ACTIVIDAD SEMANAS 10, 11 Y 12 – INFORME TÉCNICO

Manejo de bases de datos NewSQL y NoSQL en entornos Big Data: CockroachDB y MongoDB con datos abiertos de Familias en Acción

PRESENTADO POR:
DANIEL FELIPE LEÓN HIGUERA

DOCENTE: EDGAR MAURICIO LOPEZ ROJAS
ESPECIALIZACIÓN EN ANALÍTICA DE DATOS – CADI: BIG DATA
UNIVERSIDAD DE CUNDINAMARCA
CHÍA, OCTUBRE DE 2026

Tabla de contenido

1. Introducción **3**
1.1 Objetivos 3
1.2 Alcance 3
2. Metodología **3**
2.1 Entorno de trabajo y herramientas empleadas 3
2.2 Conjuntos de datos utilizados 4
3. Marco conceptual **4**
3.1 Bases de datos relacionales y no relacionales 4
3.2 Bases de datos NoSQL 5
3.3 Bases de datos NewSQL 5
3.4 Herramientas: clientes SQL y MongoDB Compass 5
3.5 Recolección de datos en Big Data 5
4. Resultados: ejercicios resueltos y validación **5**
4.1 Recolección, perfilado y limpieza de datos 6
4.2 CockroachDB (base de datos NewSQL) 6
4.3 MongoDB (base de datos NoSQL) 19
4.4 Análisis comparativo de resultados 29
4.5 Repositorio GitHub 29
4.6 Síntesis de comandos y pruebas ejecutadas 31
5. Discusión **31**
5.1 Limitaciones 31
6. Conclusiones **32**
7. Referencias **33**
Apéndice A. Scripts SQL y NoSQL utilizados **34**

1. Introducción

Los ecosistemas Big Data combinan modelos de datos distintos: las bases relacionales ofrecen consistencia transaccional pero escalan con dificultad, las NoSQL ganan flexibilidad y escalabilidad horizontal, y las NewSQL buscan unir ambas ventajas (Galindo Rosales, s. f.). Este informe documenta la práctica de las semanas 10, 11 y 12 (Universidad de Cundinamarca, 2026): ejercicios resueltos en CockroachDB (NewSQL), con Visual Studio Code como cliente SQL, sigla del lenguaje de consulta estructurado (Microsoft, s. f.) equivalente a Oracle SQL Developer (Oracle, s. f.), y en MongoDB (NoSQL), con MongoDB Compass y mongosh (MongoDB, Inc., s. f.), sobre un mismo conjunto de datos abiertos de Familias en Acción de Puerto Boyacá (Marr, 2016). Scripts, datos y evidencias están en un repositorio GitHub.

1.1 Objetivos

General: resolver ejercicios de manejo, consulta y recolección de datos en bases NewSQL y NoSQL, documentando scripts, resultados y evidencias en un repositorio GitHub. Específicos: (a) contrastar bases relacionales, NoSQL y NewSQL; (b) recolectar, perfilar y limpiar un conjunto de datos abierto; (c) diseñar y consultar un esquema en CockroachDB (transacciones, vistas, índices); (d) importar, consultar y agregar documentos en MongoDB; y (e) organizar scripts y evidencias en GitHub.

1.2 Alcance

Entorno local de pruebas, un nodo por motor en contenedores Docker. No se abordan alta disponibilidad, seguridad avanzada ni ajuste de rendimiento. No se usó Oracle Database: CockroachDB implementa el protocolo de PostgreSQL y por eso admite Visual Studio Code como cliente SQL.

2. Metodología

2.1 Entorno de trabajo y herramientas empleadas

Los motores corrieron en contenedores Docker sobre Windows 11, reutilizando los servicios de las semanas 5 y 6 (Tabla 1). La recolección y limpieza se automatizó con Python.

Tabla 1

Herramientas empleadas y su rol en la práctica

Herramienta	Tipo	Versión / despliegue	Puerto(s)
CockroachDB	NewSQL	v23.2.31, contenedor `cockroachdb-standalone`	26257 / 7080 (interfaz web)
MongoDB	NoSQL (documentos)	8.3.11, contenedor `mi-mongodb`	27017
Visual Studio Code	Cliente SQL	1.121 + extensión PostgreSQL 1.4.3	n/a
MongoDB Compass	Cliente NoSQL	Aplicación de escritorio	n/a
Python / Git	Datos y repositorio	Python 3.14; Git y GitHub	n/a

Nota. Elaboración propia.

2.2 Conjuntos de datos utilizados

Conjunto principal: “Familias focalizadas, primera fase de Familias en Acción, etapa IV, Puerto Boyacá”, un archivo CSV (valores separados por comas) de datos.gov.co con 5.543 registros y 22 columnas (género, estado, población, edad, bancarización, rangos de beneficio) (Departamento para la Prosperidad Social, 2026). Además, el archivo JSON (notación de objetos de JavaScript) del Sistema de Alerta de Bogotá (SAB), publicado por el Instituto Distrital de Gestión de Riesgos y Cambio Climático [IDIGER] (2026), para ilustrar documentos anidados y la base de subredes de salud de las semanas 5 y 6.

3. Marco conceptual

3.1 Bases de datos relacionales y no relacionales

Las relacionales usan tablas de esquema fijo y garantizan ACID (atomicidad, consistencia, aislamiento y durabilidad); las no relacionales (NoSQL) flexibilizan el esquema y parte de esas garantías para escalar horizontalmente. La Tabla 2 resume las diferencias comprobadas.

Tabla 2

Comparación entre los modelos relacional, NoSQL y NewSQL

Criterio	Relacional clásico	NoSQL (MongoDB)	NewSQL (CockroachDB)
Esquema	Fijo	Flexible por documento	Fijo, con JSONB y arreglos
Lenguaje	SQL	Consultas y *aggregation pipeline*	SQL compatible con PostgreSQL
Transacciones	ACID en un servidor	ACID por documento	ACID distribuido
Escalamiento	Vertical	Horizontal (*sharding*)	Horizontal con replicación

Nota. Elaboración propia a partir de Galindo Rosales (s. f.), MongoDB, Inc. (s. f.) y la práctica realizada.

3.2 Bases de datos NoSQL

NoSQL agrupa motores documentales, clave-valor, columnares y de grafos. MongoDB guarda documentos BSON (JSON binario) en colecciones, lo que permite estructuras anidadas sin esquema previo (MongoDB, Inc., s. f.).

3.3 Bases de datos NewSQL

NewSQL conserva el modelo relacional y SQL, y distribuye los datos en varios nodos con transacciones ACID (Galindo Rosales, s. f.). CockroachDB replica los datos en rangos con consistencia fuerte y es compatible con el protocolo de PostgreSQL (Cockroach Labs, s. f.).

3.4 Herramientas: clientes SQL y MongoDB Compass

Oracle SQL Developer es el entorno de desarrollo integrado (IDE) gratuito de Oracle para escribir y ejecutar SQL (Oracle, s. f.); aquí cumplió ese rol Visual Studio Code con la extensión PostgreSQL (Microsoft, s. f.). MongoDB Compass es la interfaz oficial de MongoDB para consultas, *pipelines* de agregación, índices y planes de ejecución (MongoDB, Inc., s. f.).

3.5 Recolección de datos en Big Data

Recolectar es identificar fuentes, obtener los datos y evaluarlos antes de usarlos. Marr (2016) reúne fuentes gratuitas de datos públicos, entre ellas portales de datos abiertos como el de este conjunto; después se perfila y limpia (sección 4.1).

4. Resultados: ejercicios resueltos y validación

El código completo de cada script está en el Apéndice A y en el repositorio; aquí se muestran su ejecución y sus resultados.

4.1 Recolección, perfilado y limpieza de datos

El script `01_recoleccion_limpieza.py` lee el CSV, lo perfila y genera un CSV limpio para MongoDB y un SQL de carga para CockroachDB. Reglas: vacíos a `ND`, textos en mayúsculas, fecha centinela `1900-01-01` a `NULL`, identificador secuencial y columnas en *snake_case*. La Figura 1 muestra el perfil: 5.543 registros, 58 celdas vacías y 1.212 fechas centinela (beneficiarios sin pagos); el 83 % son mujeres y el 46 % pertenece al Sistema de Identificación de Potenciales Beneficiarios de Programas Sociales (SISBEN).

Figura 1
Ejecución del script y perfil de calidad de los datos.

```
Terminal

PS repo-newsql-nosql-bigdata> python scripts/python/01_recoleccion_limpieza.py
Registros leidos      : 5543
Registros limpios     : 5543
Celdas vacias (antes) : 58
Fechas centinela 1900 : 1212

genero: {'MUJER': 4603, 'HOMBRE': 938, 'ND': 2}
estado: {'ACTIVO': 4826, 'NO ACTIVO': 717}
tipo_poblacion: {'SISBEN': 2559, 'UNIDOS': 1538, 'DESPLAZADOS': 993, 'ND': 453}
rango_edad: {'30-49': 2939, '50-65': 1133, '18-29': 661, '06-17': 649, '>65': 161}
bancarizado: {'SI': 4070, 'NO': 1288, 'ND': 185}
etnia: {'ND': 5069, 'AFROCOLOMBIANO - NEGRO': 401, 'ROM O GITANO': 51, 'INDIGENA': 17, 'RAIZAL': 2, 'MESTIZ
O': 2, 'PALENQUERO': 1}
```

Nota. Elaboración propia. Imagen generada a partir del script del repositorio y de la salida real de su ejecución.

4.2 CockroachDB (base de datos NewSQL)

CockroachDB corre en el contenedor `cockroachdb-standalone` (nodo único, `--insecure`). La Figura 2 muestra los contenedores activos, la versión v23.2.31 y las bases del clúster. La extensión PostgreSQL de Visual Studio Code (Figura 3) ya tenía la conexión `localhost` activa (Figura 4), cuyos parámetros resume la Tabla 3; se comprobaron además desde fuera del contenedor.

Figura 2

Contenedores activos, versión de CockroachDB y bases del clúster.

```
Terminal

PS repo-newsql-nosql-bigdata> docker ps --filter name=cockroachdb-standalone --filter name=mi-mongodb
NAMES                   IMAGE                                STATUS          PORTS
cockroachdb-standalone   cockroachdb/cockroach:latest-v23.2   Up 44 minutes   0.0.0.0:26257->26257/tcp, [::]:
26257->26257/tcp, 0.0.0.0:7080->8080/tcp, [::]:7080->8080/tcp
mi-mongodb              mongo:latest                         Up 33 minutes   0.0.0.0:27017->27017/tcp, [::]:
27017->27017/tcp
```

```
Terminal

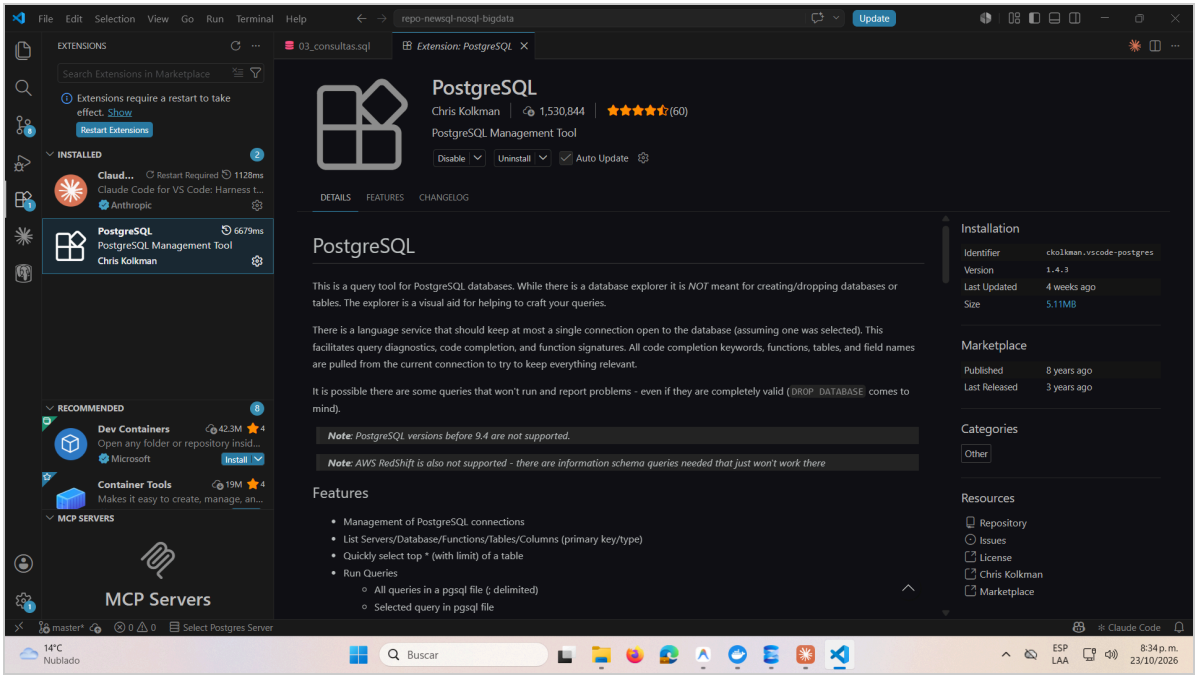
PS repo-newsql-nosql-bigdata> docker exec cockroachdb-standalone ./cockroach sql --insecure -e "SHOW DATABASE
S;"
Build Tag:        v23.2.31
Build Time:       2026/06/24 13:36:46

    database_name    | owner | primary_region | secondary_region | regions | survival_goal
---------------------+-------+----------------+------------------+---------+----------------
  defaultdb          | root  | NULL           | NULL             | {}      | NULL
  familias_accion_db | root  | NULL           | NULL             | {}      | NULL
  postgres           | root  | NULL           | NULL             | {}      | NULL
  subred_newsql_db   | root  | NULL           | NULL             | {}      | NULL
  system             | node  | NULL           | NULL             | {}      | NULL
(5 rows)
```

Nota. Elaboración propia. Imagen generada a partir del script del repositorio y de la salida real de su ejecución.

Figura 3

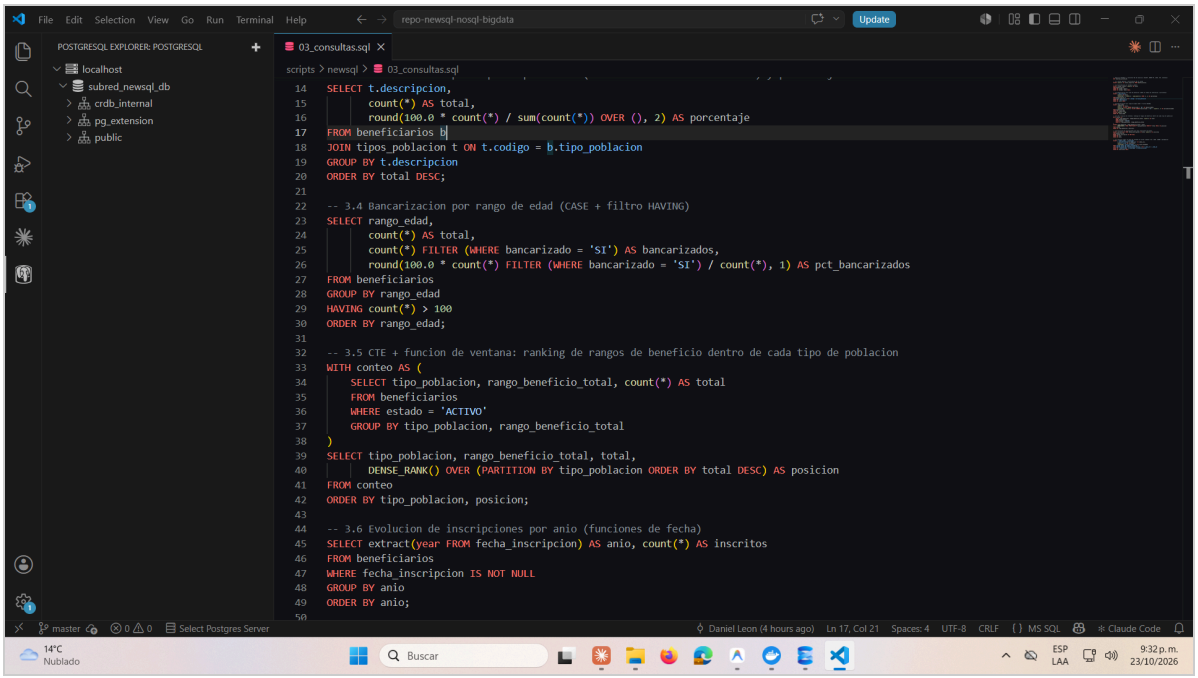
Extensión PostgreSQL instalada en Visual Studio Code.



Nota. Elaboración propia. Captura de pantalla del proceso de configuración desarrollado por el autor.

Figura 4

Conexión a CockroachDB activa en el explorador PostgreSQL.



Nota. Elaboración propia. Captura de pantalla del proceso de configuración desarrollado por el autor.

Tabla 3

Parámetros de conexión a CockroachDB desde Visual Studio Code

Parámetro	Valor
Conexión	`localhost`, puerto `26257`, usuario `root` (sin contraseña)
Base / SSL	`defaultdb` o `familias_accion_db`; estándar, sin SSL (`--insecure`)

Nota. Elaboración propia.

Ejercicio 1, esquema relacional. Modelo normalizado: tabla de hechos `beneficiarios` (claves foráneas, restricciones `CHECK` e índices secundarios), tablas de referencia `tipos_poblacion` y `rangos_edad`, y bitácora `auditoria_cambios` (identificador único universal (UUID), borrado en cascada). Se ejecutó por línea de comandos (Figura 5) y desde Visual Studio Code (Figura 6), donde el panel confirma las sentencias; luego se verificó con `information_schema` (Figura 7) que existen las cuatro tablas.

Figura 5

Ejecución del script 01_esquema.sql en CockroachDB.

```
Terminal

PS repo-newsql-nosql-bigdata> docker exec cockroachdb-standalone ./cockroach sql --insecure --echo-sql -f /tm
p/01_esquema.sql
> DROP DATABASE IF EXISTS familias_accion_db CASCADE;
DROP DATABASE
> CREATE DATABASE familias_accion_db;
CREATE DATABASE
> USE familias_accion_db;
> CREATE TABLE tipos_poblacion (
    codigo      STRING PRIMARY KEY,
    descripcion STRING NOT NULL
);
SET
CREATE TABLE
> CREATE TABLE rangos_edad (
    codigo STRING PRIMARY KEY,
    edad_min INT NOT NULL,
    edad_max INT
);
CREATE TABLE
> CREATE TABLE beneficiarios (
    id                    INT PRIMARY KEY,
    bancarizado           STRING NOT NULL CHECK (bancarizado IN ('SI','NO','ND')),
    discapacidad          STRING NOT NULL CHECK (discapacidad IN ('SI','NO','ND')),
    estado                STRING NOT NULL CHECK (estado IN ('ACTIVO','NO ACTIVO')),
    etnia                 STRING NOT NULL,
    fecha_inscripcion     DATE,
    genero                STRING NOT NULL,
    nivel_escolaridad     STRING NOT NULL,
    tipo_documento        STRING NOT NULL,
    tipo_poblacion        STRING NOT NULL REFERENCES tipos_poblacion (codigo),
    rango_beneficio_total STRING NOT NULL,
    rango_ultimo_beneficio STRING NOT NULL,
    fecha_ultimo_beneficio DATE,
    rango_edad            STRING NOT NULL REFERENCES rangos_edad (codigo),
    titular               STRING NOT NULL CHECK (titular IN ('SI','NO')),
    cantidad_beneficiarios INT NOT NULL CHECK (cantidad_beneficiarios >= 1),
    INDEX idx_estado_poblacion (estado, tipo_poblacion),
    INDEX idx_genero (genero)
);
CREATE TABLE
```

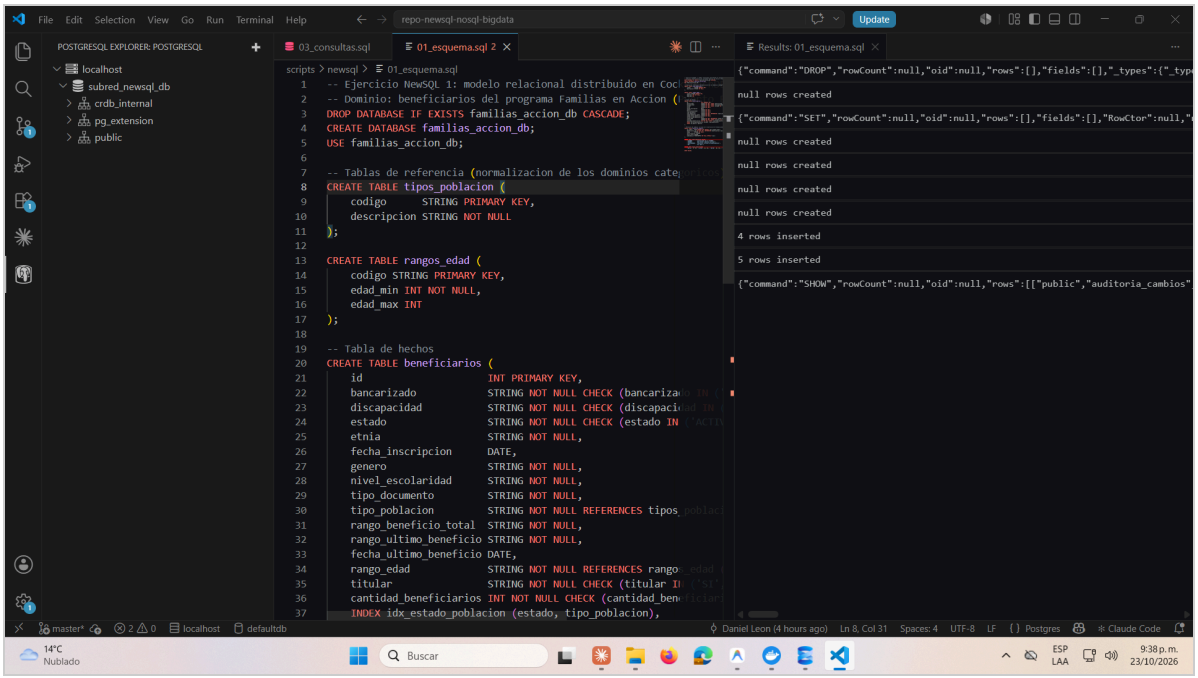
```
Terminal

PS repo-newsql-nosql-bigdata> docker exec cockroachdb-standalone ./cockroach sql --insecure --echo-sql -f /tm
p/01_esquema.sql
> CREATE TABLE auditoria_cambios (
    id_cambio   UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    beneficiario_id INT NOT NULL REFERENCES beneficiarios (id) ON DELETE CASCADE,
    campo       STRING NOT NULL,
    valor_anterior STRING,
    valor_nuevo STRING,
    cambiado_en TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE TABLE
> INSERT INTO tipos_poblacion VALUES
    ('SISBEN',      'Clasificado por el SISBEN'),
    ('UNIDOS',      'Red Unidos (pobreza extrema)'),
    ('DESPLAZADOS', 'Poblacion victima de desplazamiento'),
    ('ND',          'Sin dato');
> INSERT INTO rangos_edad VALUES
    ('06-17', 6, 17), ('18-29', 18, 29), ('30-49', 30, 49), ('50-65', 50, 65), ('>65', 66, NULL);
INSERT 0 4
INSERT 0 5
> SHOW TABLES;
  schema_name |    table_name     | type  | owner | estimated_row_count | locality
--------------+-------------------+-------+-------+---------------------+-----------
  public      | auditoria_cambios | table | root  |                   0 | NULL
  public      | beneficiarios     | table | root  |                   0 | NULL
  public      | rangos_edad       | table | root  |                   0 | NULL
  public      | tipos_poblacion   | table | root  |                   0 | NULL
(4 rows)
```

Nota. Elaboración propia. Imagen generada a partir del script del repositorio y de la salida real de su ejecución.

Figura 6

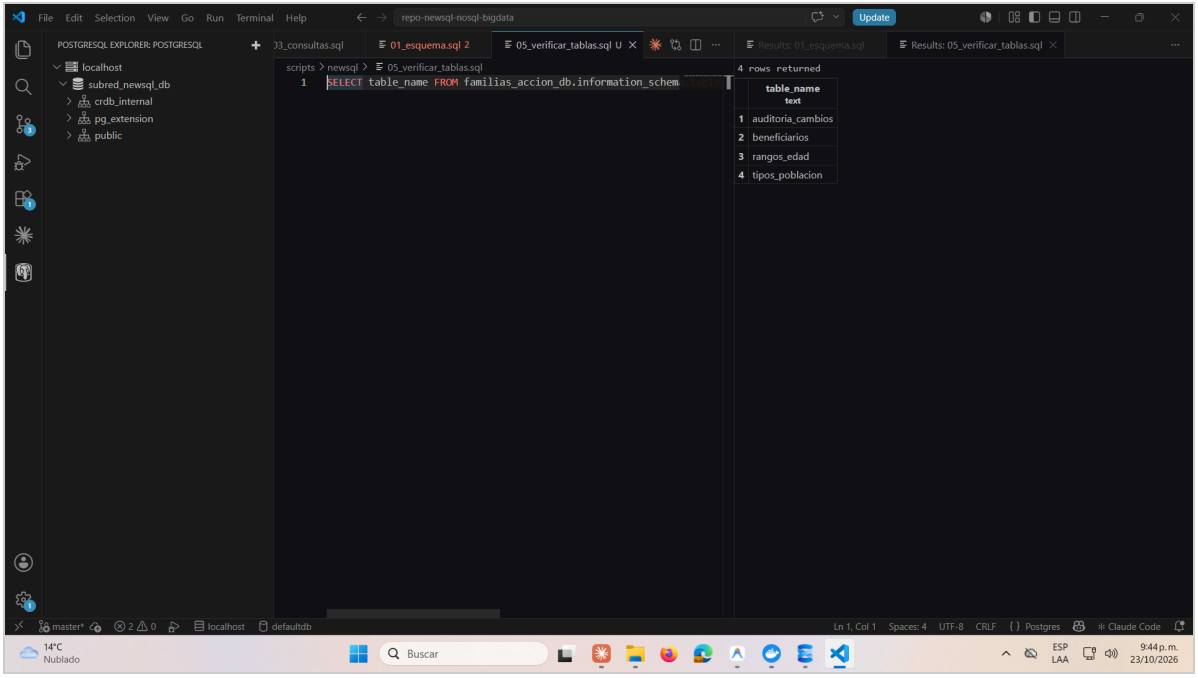
Ejecución del script 01_esquema.sql desde Visual Studio Code.



Nota. Elaboración propia. Captura de pantalla del proceso de configuración desarrollado por el autor.

Figura 7

Tablas de familias_accion_db creadas, consultadas desde information_schema.

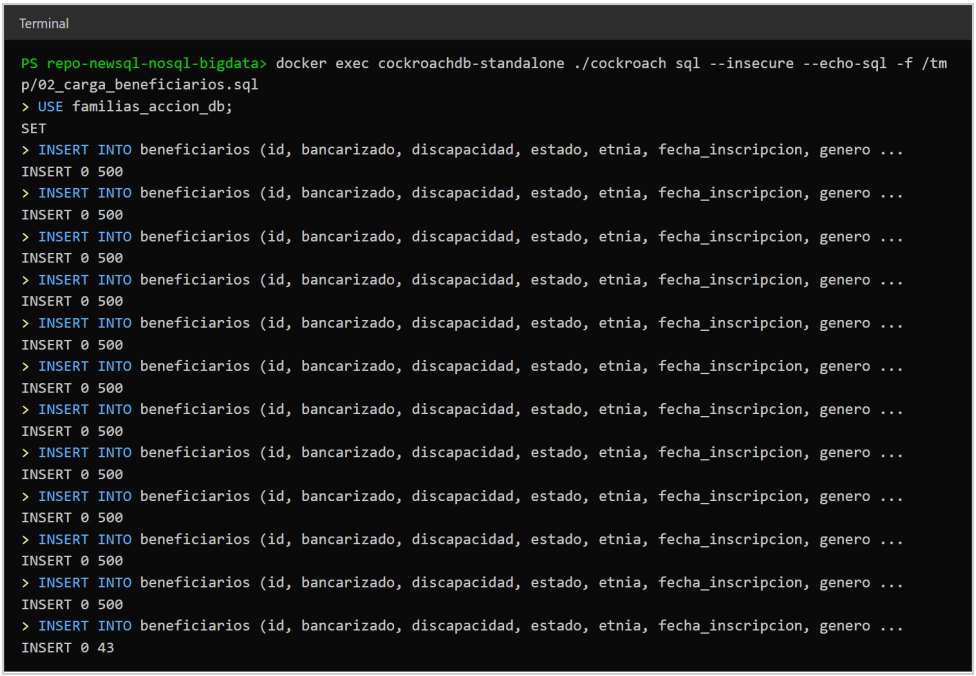


Nota. Elaboración propia. Captura de pantalla del proceso de configuración desarrollado por el autor.

Ejercicio 2, carga masiva. El script generado inserta los 5.543 registros en doce sentencias de hasta 500 filas: once lotes de 500 y uno de 43 (Figura 8 y Figura 9).

Figura 8

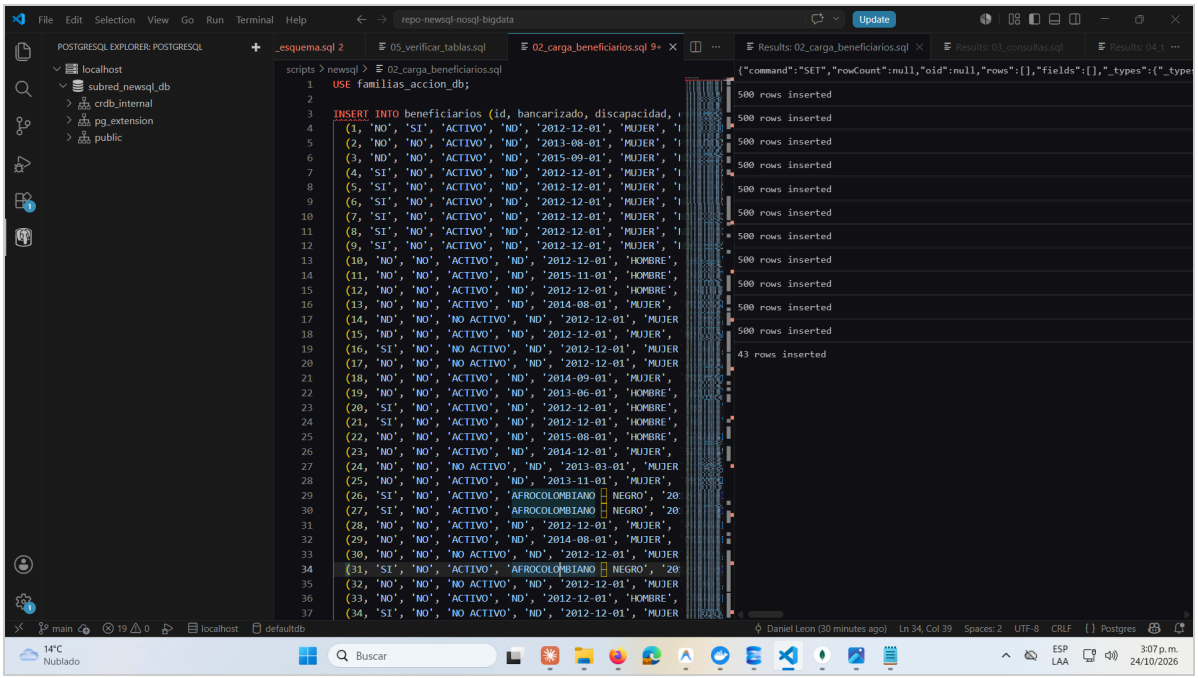
Carga por lotes de 5.543 registros en la tabla beneficiarios.



Nota. Elaboración propia. Imagen generada a partir del script del repositorio y de la salida real de su ejecución.

Figura 9

Ejecución del script 02_carga_beneficiarios.sql.

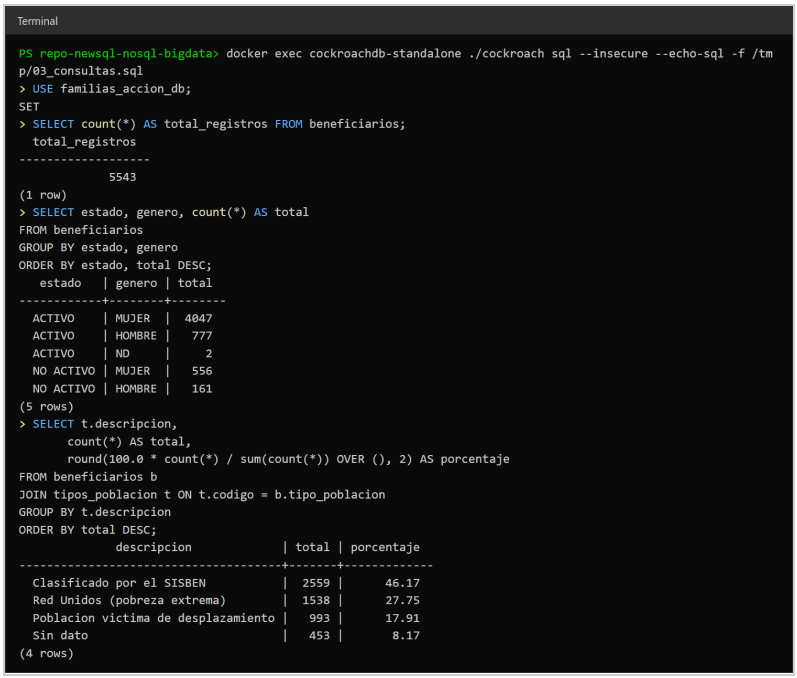


Nota. Elaboración propia. Captura de pantalla del proceso de configuración desarrollado por el autor.

Ejercicio 3, consultas de análisis. Siete consultas en `03_consultas.sql` (agrupación, `JOIN`, `FILTER`/`HAVING`, expresión de tabla común (CTE) con `DENSE_RANK()`, fechas y JSON binario (JSONB)). La Figura 10 da 5.543 registros, 4.047 mujeres y 777 hombres activos y la distribución por población (SISBEN 46,17 %, Unidos 27,75 %, desplazados 17,91 %). La Figura 11 muestra que ningún menor de 6 a 17 años está bancarizado (0 de 649), frente a más del 94 % entre 30 y 65 años. La Figura 12 clasifica los rangos de beneficio por población (17 filas), también ejecutada en Visual Studio Code (Figura 13). La Figura 14 indica que el 85 % de las inscripciones (4.739) fue en 2012 y que cada sede de salud registra 100 atenciones, con costo promedio entre $289.841 y $318.313.

Figura 10

Resultados de las consultas 3.1 a 3.3 (conteo, estado/género y población).



```
Terminal

PS repo-newsql-nosql-bigdata> docker exec cockroachdb-standalone ./cockroach sql --insecure --echo-sql -f /tm
p/03_consultas.sql
> USE familias_accion_db;
SET
> SELECT count(*) AS total_registros FROM beneficiarios;
  total_registros
-------------------
             5543
(1 row)
> SELECT estado, genero, count(*) AS total
FROM beneficiarios
GROUP BY estado, genero
ORDER BY estado, total DESC;
   estado   | genero | total
------------+--------+--------
  ACTIVO    | MUJER  |  4047
  ACTIVO    | HOMBRE |   777
  ACTIVO    | ND     |     2
  NO ACTIVO | MUJER  |   556
  NO ACTIVO | HOMBRE |   161
(5 rows)
> SELECT t.descripcion,
       count(*) AS total,
       round(100.0 * count(*) / sum(count(*)) OVER (), 2) AS porcentaje
FROM beneficiarios b
JOIN tipos_poblacion t ON t.codigo = b.tipo_poblacion
GROUP BY t.descripcion
ORDER BY total DESC;
             descripcion            | total | porcentaje
------------------------------------+-------+-------------
  Clasificado por el SISBEN         |  2559 |      46.17
  Red Unidos (pobreza extrema)      |  1538 |      27.75
  Poblacion victima de desplazamiento |  993 |      17.91
  Sin dato                          |   453 |       8.17
(4 rows)
```

Nota. Elaboración propia. Imagen generada a partir del script del repositorio y de la salida real de su ejecución.

Figura 11

Resultado de la consulta 3.4: bancarización por rango de edad.

```
Terminal

PS repo-newsql-nosql-bigdata> docker exec cockroachdb-standalone ./cockroach sql --insecure --echo-sql -f /tm
p/03_consultas.sql
> SELECT rango_edad,
       count(*) AS total,
       count(*) FILTER (WHERE bancarizado = 'SI') AS bancarizados,
       round(100.0 * count(*) FILTER (WHERE bancarizado = 'SI') / count(*), 1) AS pct_bancarizados
FROM beneficiarios
GROUP BY rango_edad
HAVING count(*) > 100
ORDER BY rango_edad;
```

Nota. Elaboración propia. Imagen generada a partir del script del repositorio y de la salida real de su ejecución.

Figura 12

Resultado de la consulta 3.5: ranking de rangos de beneficio por población.

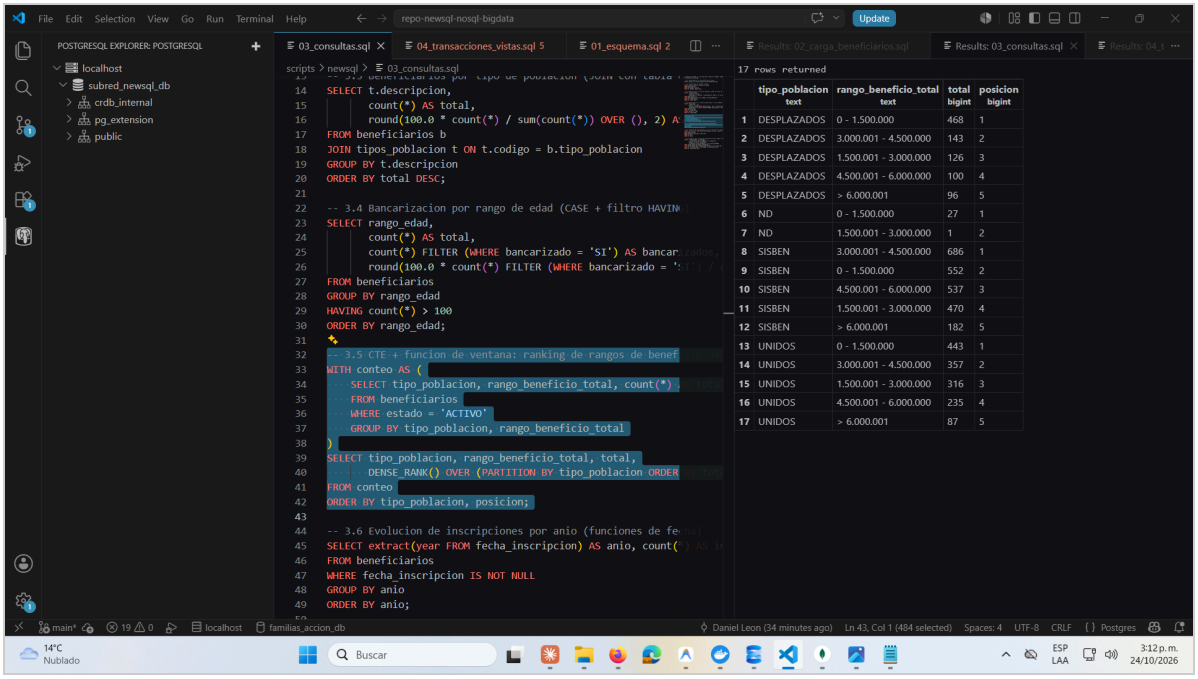
```
Terminal

PS repo-newsql-nosql-bigdata> docker exec cockroachdb-standalone ./cockroach sql --insecure --echo-sql -f /tm
p/03_consultas.sql
> WITH conteo AS (
    SELECT tipo_poblacion, rango_beneficio_total, count(*) AS total
    FROM beneficiarios
    WHERE estado = 'ACTIVO'
    GROUP BY tipo_poblacion, rango_beneficio_total
)
SELECT tipo_poblacion, rango_beneficio_total, total,
       DENSE_RANK() OVER (PARTITION BY tipo_poblacion ORDER BY total DESC) AS posicion
FROM conteo
ORDER BY tipo_poblacion, posicion;
  rango_edad | total | bancarizados | pct_bancarizados
-------------+-------+--------------+-------------------
  06-17      |   649 |            0 |              0.0
  18-29      |   661 |           83 |             12.6
  30-49      |  2939 |         2763 |             94.0
  50-65      |  1133 |         1081 |             95.4
  >65        |   161 |          143 |             88.8
(5 rows)
  tipo_poblacion | rango_beneficio_total | total | posicion
-----------------+-----------------------+-------+-----------
  DESPLAZADOS    | 0 - 1.500.000         |   468 |        1
  DESPLAZADOS    | 3.000.001 - 4.500.000 |   143 |        2
  DESPLAZADOS    | 1.500.001 - 3.000.000 |   126 |        3
  DESPLAZADOS    | 4.500.001 - 6.000.000 |   100 |        4
  DESPLAZADOS    | > 6.000.001           |    96 |        5
  ND             | 0 - 1.500.000         |    27 |        1
  ND             | 1.500.001 - 3.000.000 |     1 |        2
  SISBEN         | 3.000.001 - 4.500.000 |   686 |        1
  SISBEN         | 0 - 1.500.000         |   552 |        2
  SISBEN         | 4.500.001 - 6.000.000 |   537 |        3
  SISBEN         | 1.500.001 - 3.000.000 |   470 |        4
  SISBEN         | > 6.000.001           |   182 |        5
  UNIDOS         | 0 - 1.500.000         |   443 |        1
  UNIDOS         | 3.000.001 - 4.500.000 |   357 |        2
```

Nota. Elaboración propia. Imagen generada a partir del script del repositorio y de la salida real de su ejecución.

Figura 13

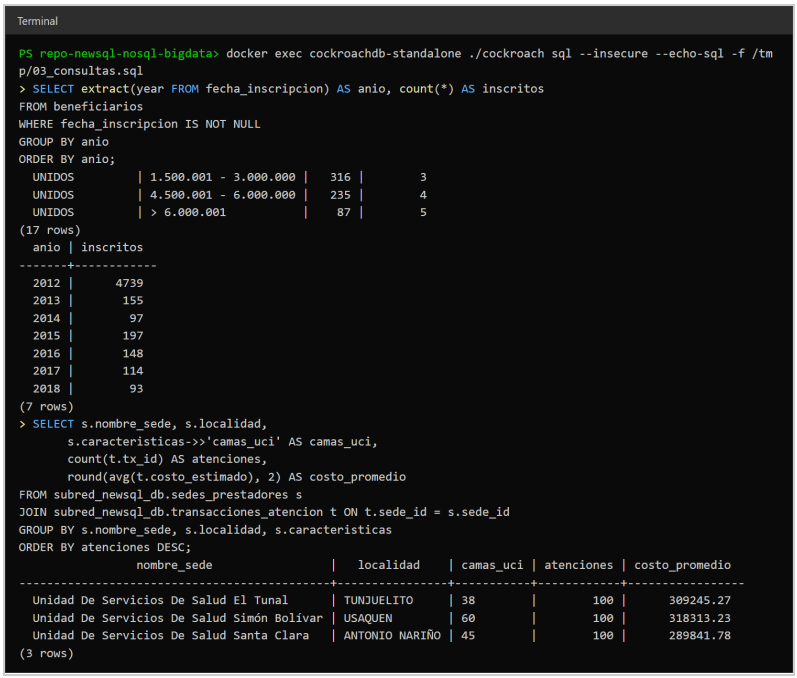
Consulta 3.5 (CTE y DENSE_RANK) ejecutada en Visual Studio Code.



Nota. Elaboración propia. Captura de pantalla del proceso de configuración desarrollado por el autor.

Figura 14

Resultados de las consultas 3.6 y 3.7 (inscripciones por año y subredes).



Nota. Elaboración propia. Imagen generada a partir del script del repositorio y de la salida real de su ejecución.

Ejercicio 4, transacciones, vista e índices. Una transacción con `COMMIT` reactiva al beneficiario 14 y lo registra en la bitácora; otra deshace con `ROLLBACK` un cambio a 99 integrantes que solo fue visible dentro de ella y dejó el valor original (1) (Figura 15; la primera también desde Visual Studio Code, Figura 16). Una vista resume por población y estado, y el `EXPLAIN` usa el índice `idx_estado_poblacion` sin barrido completo; la tabla ocupa un solo rango con una réplica, propio de un nodo único (Figura 17).

Figura 15

Transacción confirmada (COMMIT) y transacción revertida (ROLLBACK).

```
Terminal

PS repo-newsql-nosql-bigdata> docker exec cockroachdb-standalone ./cockroach sql --insecure -f /tmp/04_transa
cciones_vistas.sql
SET
  id |  estado
-----+------------
  14 | NO ACTIVO
(1 row)
BEGIN
INSERT 0 1
UPDATE 1
COMMIT
  id | estado | campo  | valor_anterior | valor_nuevo
-----+--------+--------+----------------+-------------
  14 | ACTIVO | estado | NO ACTIVO      | ACTIVO
(1 row)
```

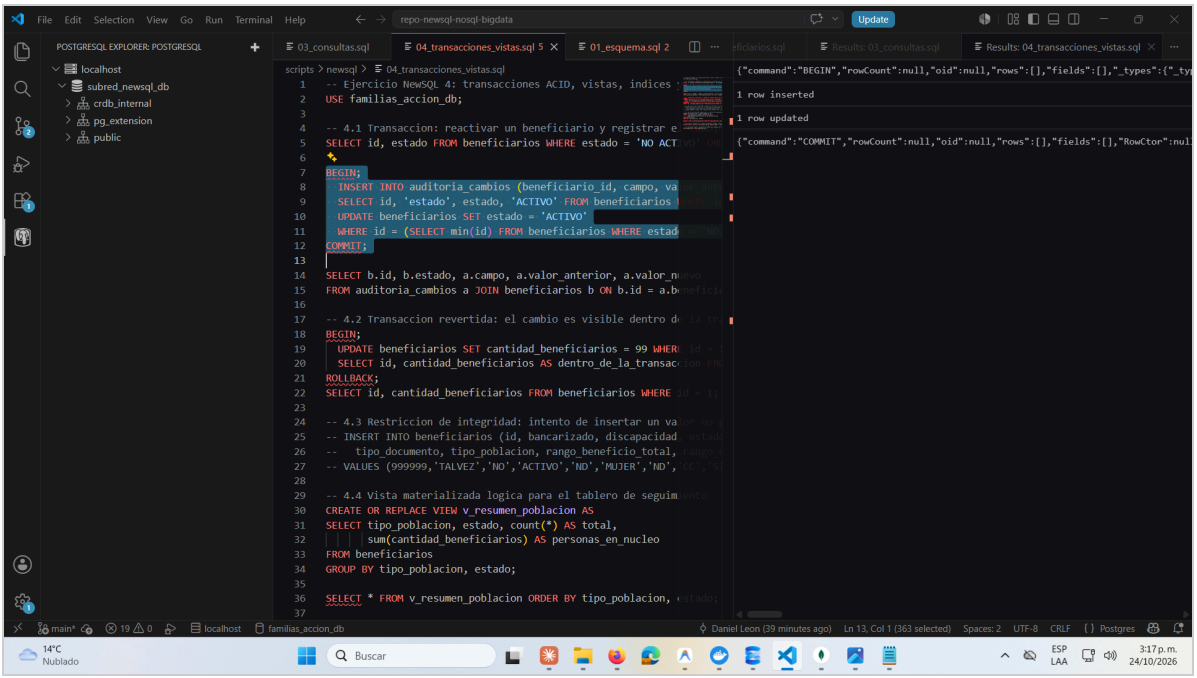
```
Terminal

PS repo-newsql-nosql-bigdata> docker exec cockroachdb-standalone ./cockroach sql --insecure -f /tmp/04_transa
cciones_vistas.sql
BEGIN
UPDATE 1
  id | dentro_de_la_transaccion
-----+--------------------------
   1 |                       99
(1 row)
ROLLBACK
  id | cantidad_beneficiarios
-----+------------------------
   1 |                      1
(1 row)
```

Nota. Elaboración propia. Imagen generada a partir del script del repositorio y de la salida real de su ejecución.

Figura 16

Transacción con COMMIT ejecutada en Visual Studio Code.



Nota. Elaboración propia. Captura de pantalla del proceso de configuración desarrollado por el autor.

Figura 17

Vista v_resumen_poblacion, plan de ejecución (EXPLAIN) y rangos de la tabla.

```
Terminal

PS repo-newsql-nosql-bigdata> docker exec cockroachdb-standalone ./cockroach sql --insecure -f /tmp/04_transa
cciones_vistas.sql
CREATE VIEW
  tipo_poblacion |  estado   | total | personas_en_nucleo
-----------------+-----------+-------+---------------------
  DESPLAZADOS    | ACTIVO    |   933 |               1610
  DESPLAZADOS    | NO ACTIVO |    60 |                163
  ND             | ACTIVO    |    28 |                 37
  ND             | NO ACTIVO |   425 |               1348
  SISBEN         | ACTIVO    |  2427 |               6209
  SISBEN         | NO ACTIVO |   132 |                730
  UNIDOS         | ACTIVO    |  1439 |               3463
  UNIDOS         | NO ACTIVO |    99 |                524
(8 rows)
```

```
Terminal

PS repo-newsql-nosql-bigdata> docker exec cockroachdb-standalone ./cockroach sql --insecure -f /tmp/04_transa
cciones_vistas.sql
                        info
------------------------------------------------------
  distribution: local
  vectorized: true

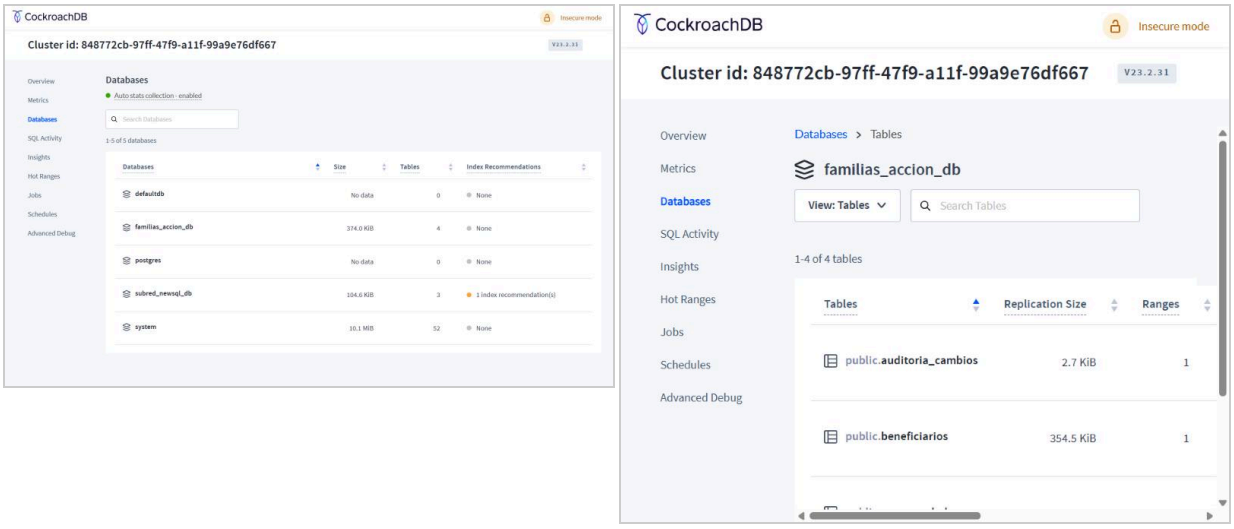
  • group (scalar)
  │
  └── • scan
        missing stats
        table: beneficiarios@idx_estado_poblacion
        spans: [/'ACTIVO'/'SISBEN' - /'ACTIVO'/'SISBEN']
(9 rows)
      start_key      |   end_key    | range_id | replicas | replica_localities | voting_replicas | non_votin
g_replicas | learner_replicas | split_enforced_until
---------------------+--------------+----------+----------+--------------------+-----------------+----------
-----------+------------------+----------------------
  <before:/Table/111> | <after:/Max> |       68 | {1}      | {""}               | {1}             | {}
| {}               | NULL
(1 row)
```

Nota. Elaboración propia. Imagen generada a partir del script del repositorio y de la salida real de su ejecución.

Validación en el Admin UI (puerto 7080). La Figura 18 muestra las bases del nodo y las cuatro tablas de `familias_accion_db`; la Figura 19, el detalle de `beneficiarios` con sus tres índices y los trabajos del clúster en estado SUCCEEDED.

Figura 18

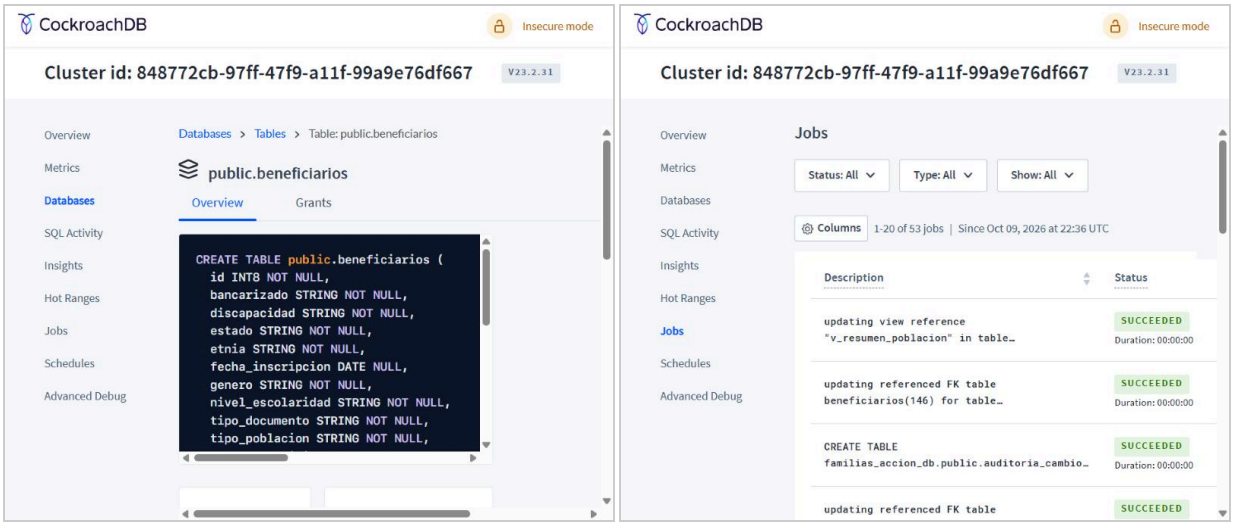
Admin UI: bases de datos del clúster y tablas de familias_accion_db.



Nota. Elaboración propia. Captura de pantalla del proceso de configuración desarrollado por el autor.

Figura 19

Admin UI: detalle de la tabla beneficiarios y trabajos del clúster.



Nota. Elaboración propia. Captura de pantalla del proceso de configuración desarrollado por el autor.

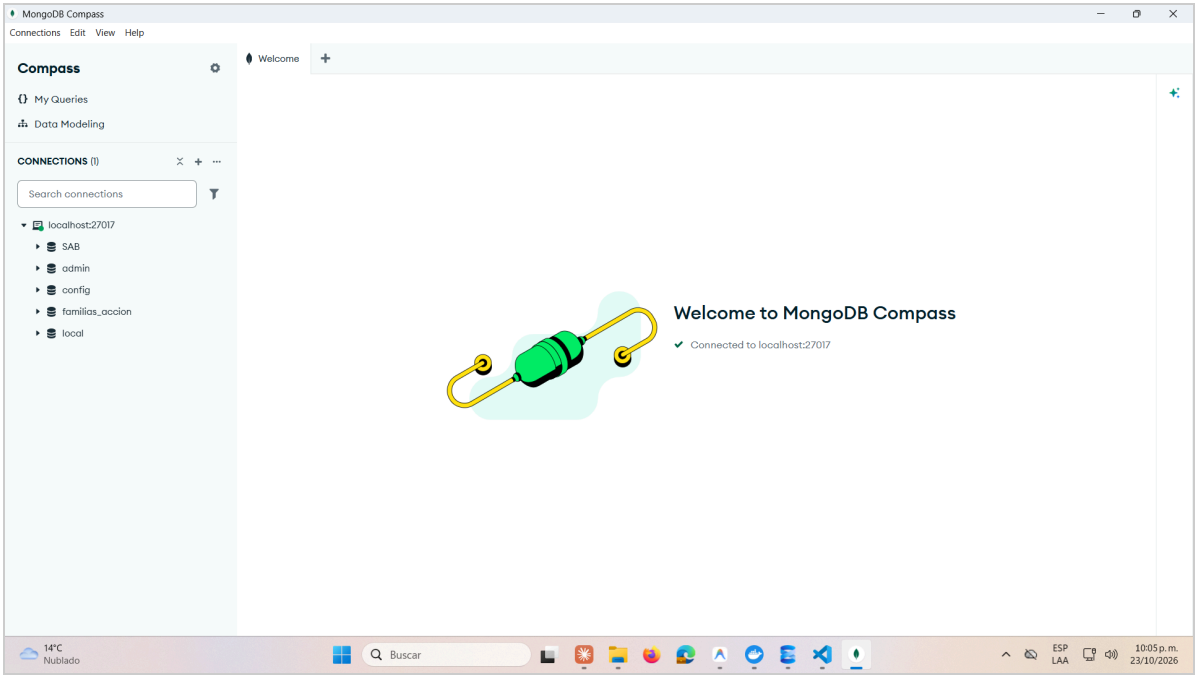
4.3 MongoDB (base de datos NoSQL)

MongoDB 8.3.11 corre en el contenedor `mi-mongodb`. Compass se conectó a `localhost:27017` con autenticación contra `admin` (sin guardar credenciales en el repositorio: los scripts las toman del contenedor) y lista las bases `familias_accion` y `SAB` (Figura 20). El CSV limpio se importó con `mongoimport --ignoreBlanks` y el JSON del SAB como un documento con

72 sensores: 5.543 documentos sin fallos (Figura 21). La colección tiene 5.543 documentos y tres índices (Figura 22).

Figura 20

Conexión a localhost:27017 en MongoDB Compass con las bases familias_accion y SAB.



Nota. Elaboración propia. Captura de pantalla del proceso de configuración desarrollado por el autor.

Figura 21

Versión del servidor e importación de beneficiarios y del documento del SAB.

```
Terminal

PS repo-newsql-nosql-bigdata> docker exec mi-mongodb mongod --version
db version v8.3.11
```

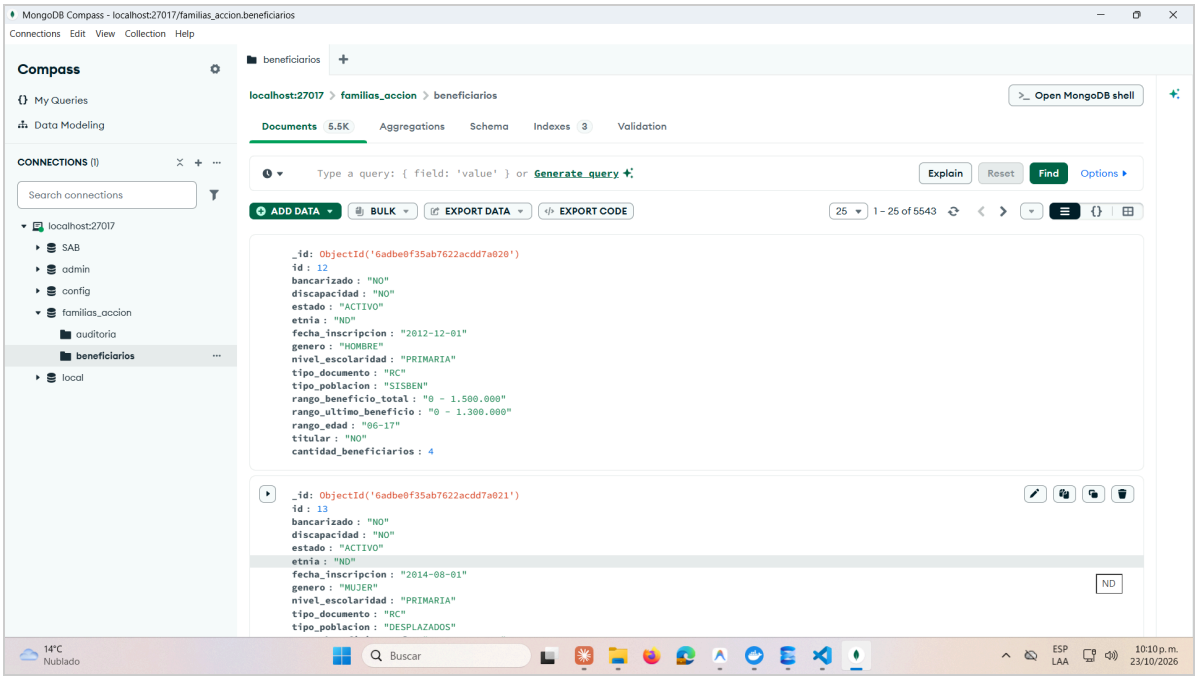
```
Terminal

PS repo-newsql-nosql-bigdata> bash scripts/nosql/01_importar.sh
2026-10-23T22:34:27.762+0000    2026-10-23T22:34:27.764+0000    dropping: `familias_accion.beneficiarios`
2026-10-23T22:34:28.033+0000    5543 document(s) imported successfully. 0 document(s) failed to import.
2026-10-23T22:34:28.468+0000    2026-10-23T22:34:28.468+0000    dropping: `SAB.sensores`
2026-10-23T22:34:28.491+0000    1 document(s) imported successfully. 0 document(s) failed to import.
```

Nota. Elaboración propia. Imagen generada a partir del script del repositorio y de la salida real de su ejecución.

Figura 22

Colección familias_accion.beneficiarios con 5.543 documentos.



Nota. Elaboración propia. Captura de pantalla del proceso de configuración desarrollado por el autor.

Ejercicio 1, creación, lectura, actualización y borrado (CRUD) y consultas (`02_crud_consultas.js`). Consultas con filtro, proyección y operadores `$or`, `$in`, `$gt`: hay 71 personas con discapacidad o de etnia indígena y 123 núcleos de más de tres integrantes en población Unidos o desplazada. `insertOne`, `updateOne` y `deleteOne` operan sobre un documento de prueba con un campo anidado inexistente en el resto (esquema flexible), y `updateMany` modifica 6 documentos (Figura 23). El filtro equivale al de la pestaña Documents de Compass (Figura 24, 2.463 resultados) y la shell mongosh integrada devuelve 2.427 para SISBEN activa (Figura 25).

Figura 23

Consultas y operaciones CRUD en mongosh.

```
Terminal

PS repo-newsql-nosql-bigdata> docker exec mi-mongodb mongosh -u $MONGO_INITDB_ROOT_USERNAME -p *** --authenti
cationDatabase admin --quiet --file /tmp/02_crud_consultas.js
--- 1.1 Total de documentos y un documento de ejemplo
5543
{
  id: 12, bancarizado: 'NO', discapacidad: 'NO', estado: 'ACTIVO', etnia: 'ND', fecha_inscripcion: '2012-12-0
1', genero: 'HOMBRE', nivel_escolaridad: 'PRIMARIA', tipo_documento: 'RC', tipo_poblacion: 'SISBEN', rango_be
neficio_total: '0 - 1.500.000', rango_ultimo_beneficio: '0 - 1.300.000', rango_edad: '06-17', titular: 'NO',
cantidad_beneficiarios: 4
}
--- 1.2 find con filtro y proyeccion: mujeres titulares, activas, de 30-49 anios
[
  { id: 3, tipo_poblacion: 'DESPLAZADOS', rango_beneficio_total: '3.000.001 - 4.500.000' },
  { id: 43, tipo_poblacion: 'SISBEN', rango_beneficio_total: '1.500.001 - 3.000.000' },
  { id: 47, tipo_poblacion: 'DESPLAZADOS', rango_beneficio_total: '1.500.001 - 3.000.000' }
]
```

```
Terminal

PS repo-newsql-nosql-bigdata> docker exec mi-mongodb mongosh -u $MONGO_INITDB_ROOT_USERNAME -p *** --authenti
cationDatabase admin --quiet --file /tmp/02_crud_consultas.js
--- 1.3 Operadores: personas con discapacidad O etnia indigena
71
--- 1.4 $in y $gt: nucleos con mas de 3 integrantes en poblacion UNIDOS o DESPLAZADOS
123
--- 1.5 insertOne / updateOne / deleteOne
{
  id: 999999, estado: 'ACTIVO', genero: 'MUJER', tipo_poblacion: 'SISBEN', nota: 'documento de prueba (esquem
a flexible)', contacto: { telefono: '3000000000' }
}
1
1
```

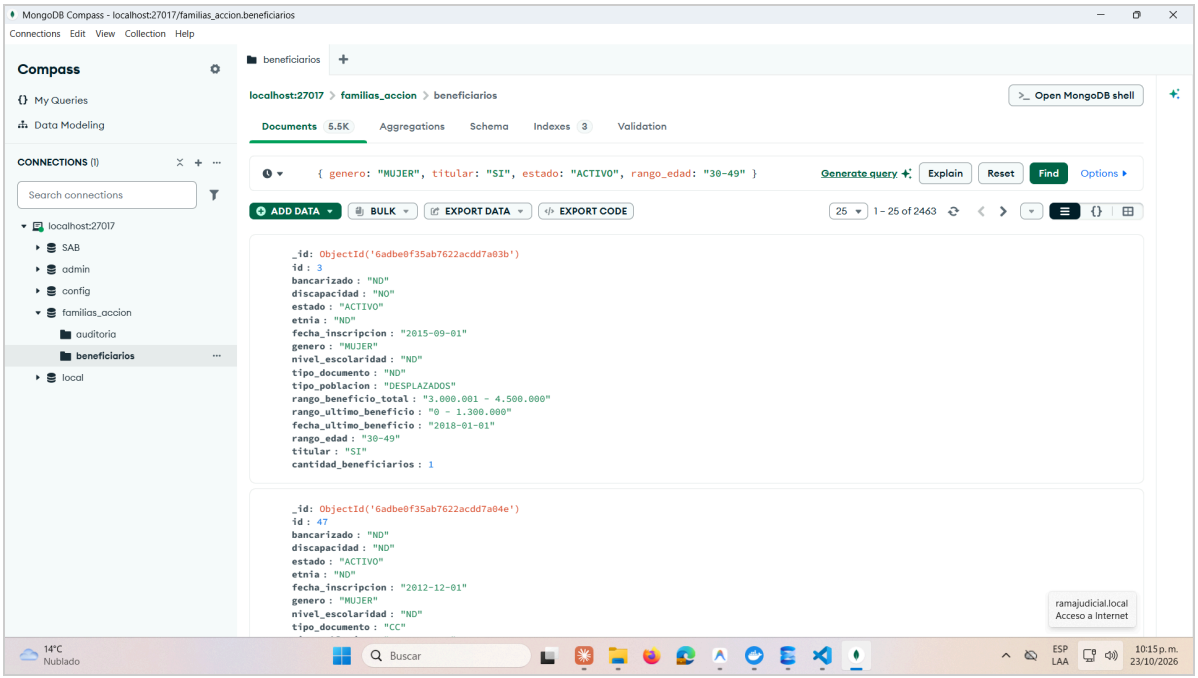
```
Terminal

PS repo-newsql-nosql-bigdata> docker exec mi-mongodb mongosh -u $MONGO_INITDB_ROOT_USERNAME -p *** --authenti
cationDatabase admin --quiet --file /tmp/02_crud_consultas.js
--- 1.6 updateMany: normalizar tipo_documento 'NO DEFINIDO' a 'ND'
6
--- 1.7 Coleccion SAB.sensores (documentos con arreglo anidado)
[
  { TipoSensores: [ { IDSENSOR: 15, IDESTACION: 1, ESTACION: 'COLMAC', IDTIPOSENSOR: 5, TIPOSENSOR: 'Pluviome
tro', UNIDADMEDIDA: 'Milimetros', LATITUD: 4.8132608056, LONGITUD: -74.0314564148, ESTADO: 1, IDTIPOTECNOLOGI
A: 2, TIPOTECNOLOGIA: 'Campbell', IDLOCALIDAD: 1, LOCALIDAD: 'USAQUEN', VISIBLE: 1, ACUMULADODIA: 0, VALORLEC
TURA: 0, FECHALECTURA: '2026-09-22 11:51:00.0', ALTITUD: 0, CODIGO: null, GEOLOGIA: null, IDESTADOSSISMOS: 8,
ESTADOESTACION: 0, PRESIONATMOSFERICA: null } ] }
]
...
```

Nota. Elaboración propia. Imagen generada a partir del script del repositorio y de la salida real de su ejecución.

Figura 24

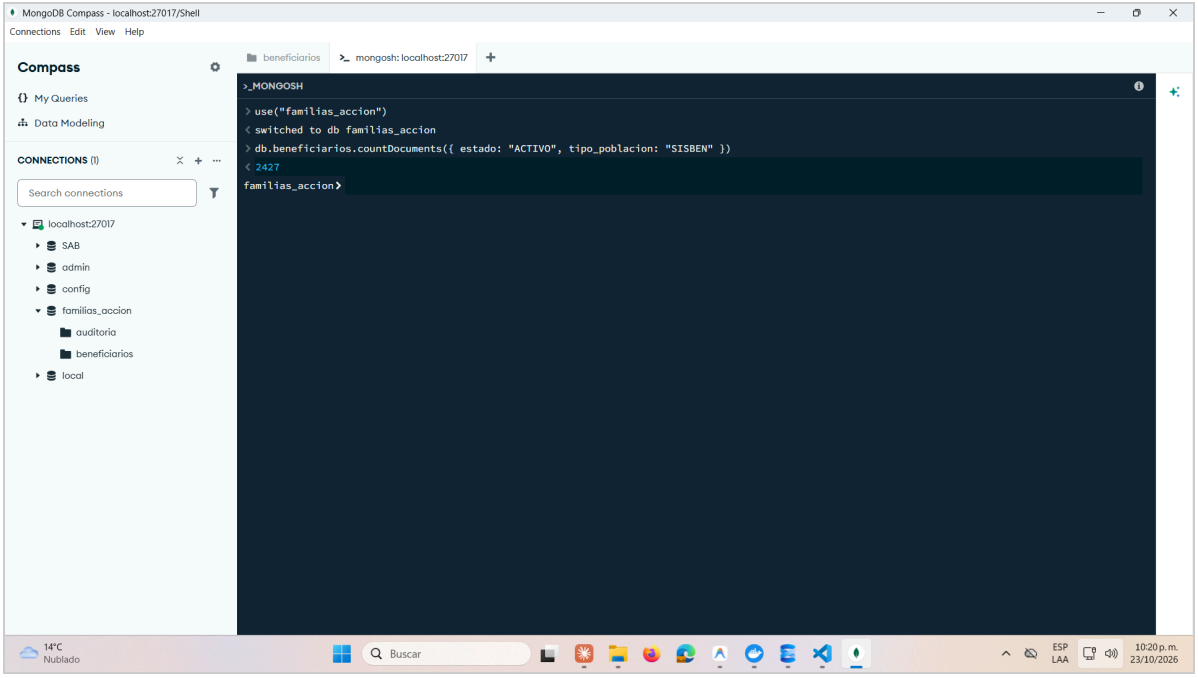
Consulta con filtro en la pestaña Documents de Compass.



Nota. Elaboración propia. Captura de pantalla del proceso de configuración desarrollado por el autor.

Figura 25

Consulta equivalente ejecutada en la shell mongosh integrada en Compass.



Nota. Elaboración propia. Captura de pantalla del proceso de configuración desarrollado por el autor.

Ejercicio 2, agregación (`03_agregaciones.js`). Reproduce los análisis de SQL con `$group`, `$project`, `$bucket`, `$facet` y `$unwind`: la población por estado coincide con la vista SQL (Figura 26), los porcentajes de bancarización son los mismos (0 % en menores, 94 % de 30 a 49 años), 4.861 núcleos tienen un solo integrante, y Ciudad Bolívar tiene más sensores del SAB (11), seguida de Usme (9) y Usaquén (8) (Figura 27). Las figuras Figura 28 y Figura 29 muestran el pipeline 2.2 construido por etapas en Compass.

Figura 26

Resultado de la agregación 2.1: población por estado.

```
Terminal

PS repo-newsql-nosql-bigdata> docker exec mi-mongodb mongosh -u $MONGO_INITDB_ROOT_USERNAME -p *** --authenti
cationDatabase admin --quiet --file /tmp/03_agregaciones.js
--- 2.1 $group: beneficiarios por tipo de poblacion y estado
[
  { _id: { poblacion: 'DESPLAZADOS', estado: 'ACTIVO' }, total: 933, personas: 1610 },
  { _id: { poblacion: 'DESPLAZADOS', estado: 'NO ACTIVO' }, total: 60, personas: 163 },
  { _id: { poblacion: 'ND', estado: 'ACTIVO' }, total: 28, personas: 37 },
  { _id: { poblacion: 'ND', estado: 'NO ACTIVO' }, total: 425, personas: 1348 },
  { _id: { poblacion: 'SISBEN', estado: 'ACTIVO' }, total: 2427, personas: 6209 },
  { _id: { poblacion: 'SISBEN', estado: 'NO ACTIVO' }, total: 132, personas: 730 },
  { _id: { poblacion: 'UNIDOS', estado: 'ACTIVO' }, total: 1438, personas: 3462 },
  { _id: { poblacion: 'UNIDOS', estado: 'NO ACTIVO' }, total: 100, personas: 525 }
]
```

Nota. Elaboración propia. Imagen generada a partir del script del repositorio y de la salida real de su ejecución.

Figura 27

Resultado de las agregaciones 2.2 a 2.6.

```
Terminal

PS repo-newsql-nosql-bigdata> docker exec mi-mongodb mongosh -u $MONGO_INITDB_ROOT_USERNAME -p *** --authenti
cationDatabase admin --quiet --file /tmp/03_agregaciones.js
--- 2.2 $match + $group + $project: % bancarizados por rango de edad
[
  { _id: '06-17', total: 649, bancarizados: 0, pct: 0 },
  { _id: '18-29', total: 661, bancarizados: 83, pct: 12.6 },
  { _id: '30-49', total: 2939, bancarizados: 2763, pct: 94 },
  { _id: '50-65', total: 1133, bancarizados: 1081, pct: 95.4 },
  { _id: '>65', total: 161, bancarizados: 143, pct: 88.8 }
]
--- 2.3 $bucket: nucleos por tamanio
[
  { _id: 1, nucleos: 4861 },
  { _id: 2, nucleos: 286 },
  { _id: 3, nucleos: 178 },
  { _id: 5, nucleos: 111 },
  { _id: 10, nucleos: 87 },
  { _id: 'otros', nucleos: 20 }
]
```

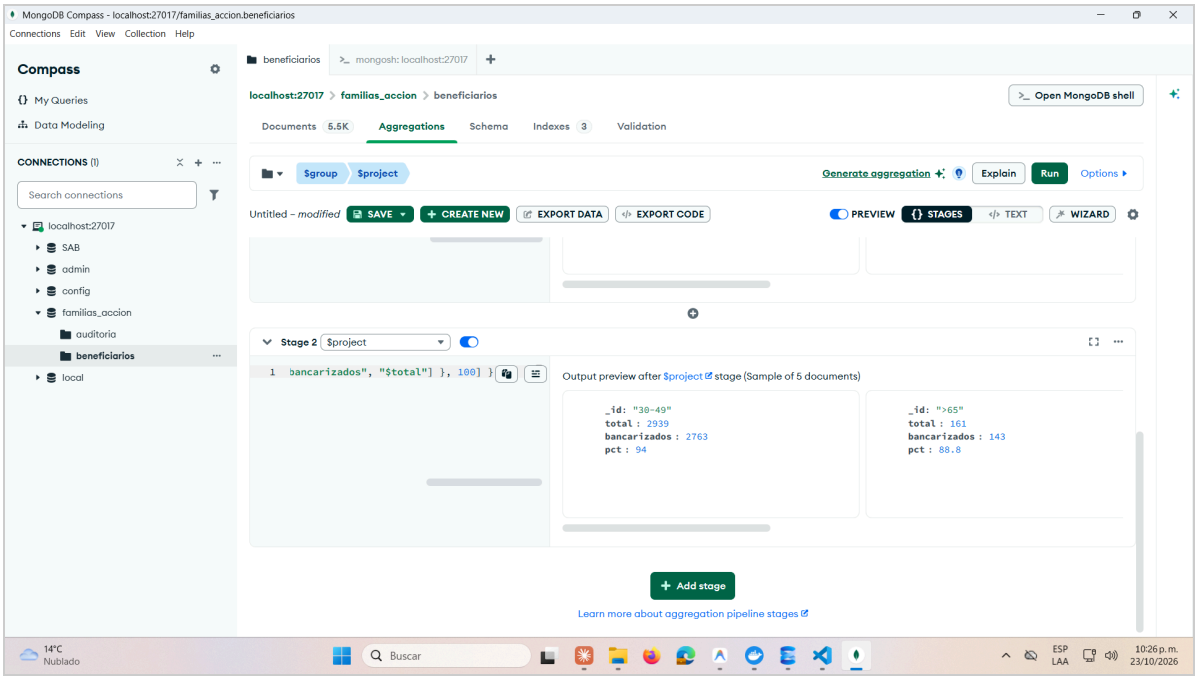
```
Terminal

PS repo-newsql-nosql-bigdata> docker exec mi-mongodb mongosh -u $MONGO_INITDB_ROOT_USERNAME -p *** --authenti
cationDatabase admin --quiet --file /tmp/03_agregaciones.js
--- 2.4 $group por anio de inscripcion (cadena ISO -> substring)
[
  { _id: '2012', inscritos: 4739 },
  { _id: '2013', inscritos: 155 },
  { _id: '2014', inscritos: 97 },
  { _id: '2015', inscritos: 197 },
  { _id: '2016', inscritos: 148 },
  { _id: '2017', inscritos: 114 },
  { _id: '2018', inscritos: 93 }
]
--- 2.5 $facet: varios resumenes en una sola consulta
[
  { porGenero: [ { _id: 'ND', n: 2 },
  { _id: 'HOMBRE', n: 938 },
  { _id: 'MUJER', n: 4603 } ], porEtnia: [ { _id: 'ND', n: 5069 },
  { _id: 'AFROCOLOMBIANO - NEGRO', n: 401 },
  { _id: 'ROM O GITANO', n: 51 } ] }
]
--- 2.6 SAB: $unwind + $group por localidad (sensores)
[
  { _id: 'CIUDAD BOLIVAR', sensores: 11 },
  { _id: 'USME', sensores: 9 },
  { _id: 'USAQUEN', sensores: 8 },
  { _id: 'SAN CRISTOBAL', sensores: 7 },
  { _id: 'SUBA', sensores: 6 }
]
```

Nota. Elaboración propia. Imagen generada a partir del script del repositorio y de la salida real de su ejecución.

Figura 28

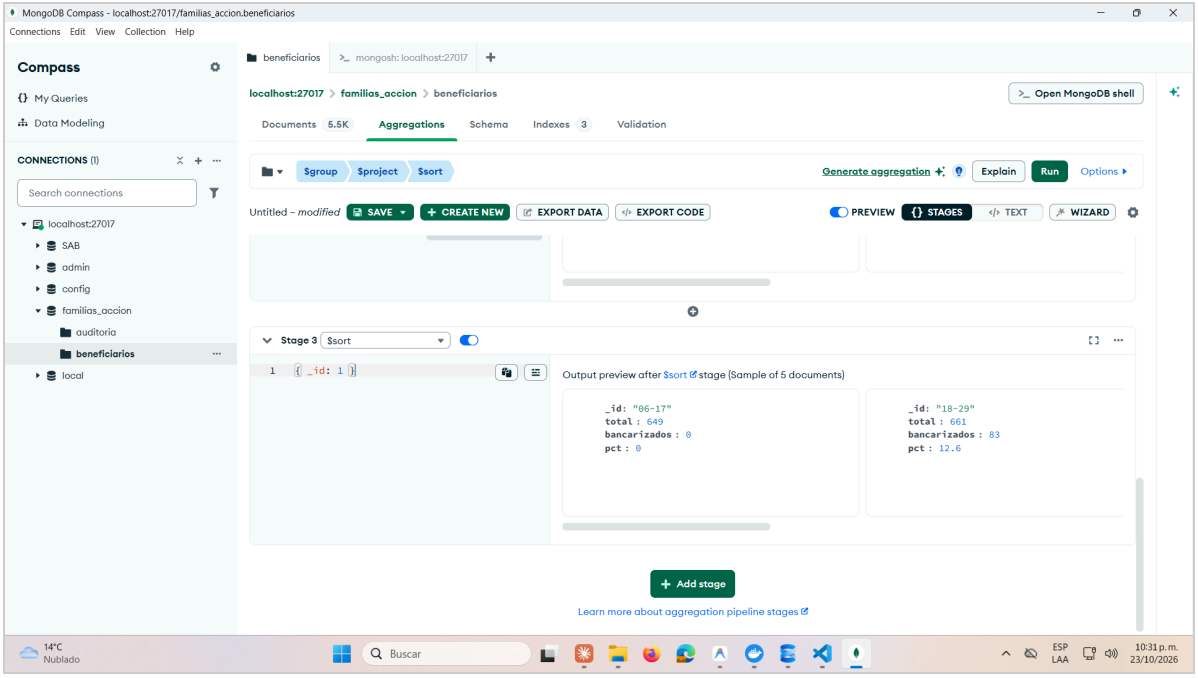
Pipeline de agregación: etapas $group y $project.



Nota. Elaboración propia. Captura de pantalla del proceso de configuración desarrollado por el autor.

Figura 29

Resultado del pipeline con la etapa $sort.



Nota. Elaboración propia. Captura de pantalla del proceso de configuración desarrollado por el autor.

Ejercicio 3, índices y validación (`04_indices_rendimiento.js`). Sin índice la consulta recorrió 5.543 documentos (`COLLSCAN`) para devolver 2.427; con el índice compuesto (`IXSCAN`) examinó solo esos 2.427. Un validador `$jsonSchema` rechazó un documento inválido (código 121) y aceptó uno válido (Figura 30). Compass lo confirma en Indexes (Figura 31) y en el Explain Plan (Figura 32).

Figura 30

Plan de ejecución con y sin índice, índices existentes y validador de esquema.

```
Terminal

PS repo-newsql-nosql-bigdata> docker exec mi-mongodb mongosh -u $MONGO_INITDB_ROOT_USERNAME -p *** --authenti
cationDatabase admin --quiet --file /tmp/04_indices_rendimiento.js
--- 3.1 Sin indice (COLLSCAN)
{
  etapas: 'COLLSCAN', docsExaminados: 5543, devueltos: 2427
}
--- 3.2 Creacion de indice compuesto
idx_estado_poblacion
idx_id_unico
--- 3.3 Con indice (IXSCAN)
{
  etapas: 'FETCH > IXSCAN', docsExaminados: 2427, devueltos: 2427
}
```

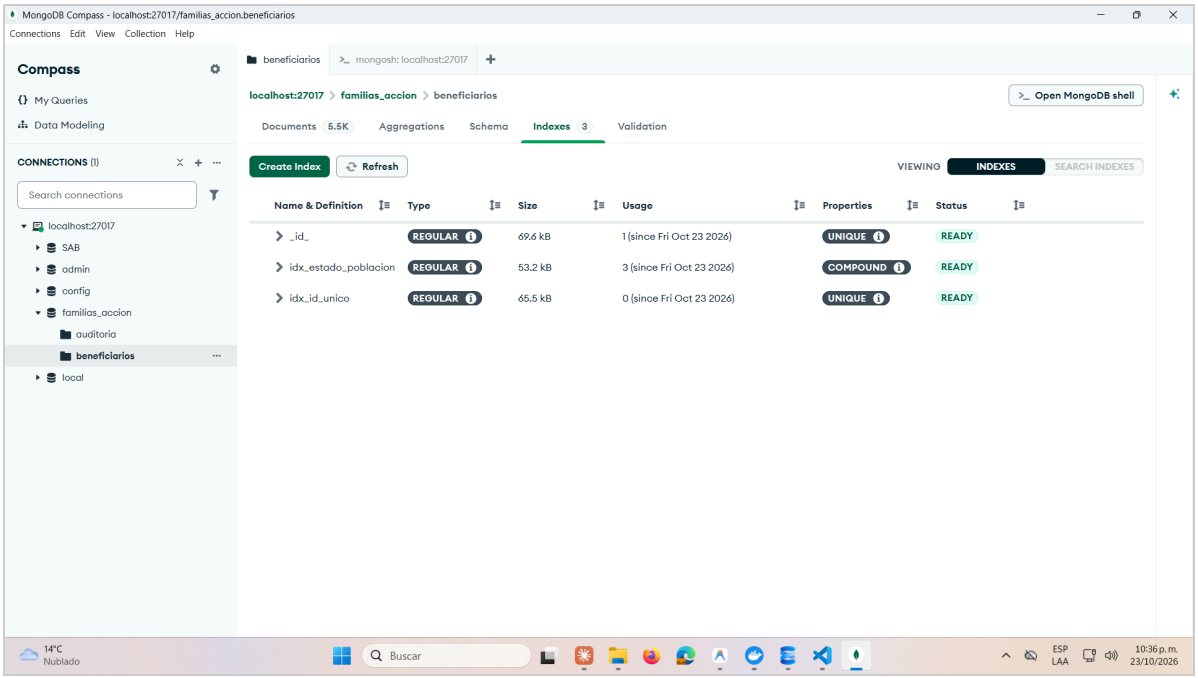
```
Terminal

PS repo-newsql-nosql-bigdata> docker exec mi-mongodb mongosh -u $MONGO_INITDB_ROOT_USERNAME -p *** --authenti
cationDatabase admin --quiet --file /tmp/04_indices_rendimiento.js
--- 3.4 Indices existentes
[
  '_id_', 'idx_estado_poblacion', 'idx_id_unico'
]
--- 3.5 Validacion de esquema opcional ($jsonSchema) en una coleccion nueva
rechazado por validador (codigo 121): Document failed validation
true
```

Nota. Elaboración propia. Imagen generada a partir del script del repositorio y de la salida real de su ejecución.

Figura 31

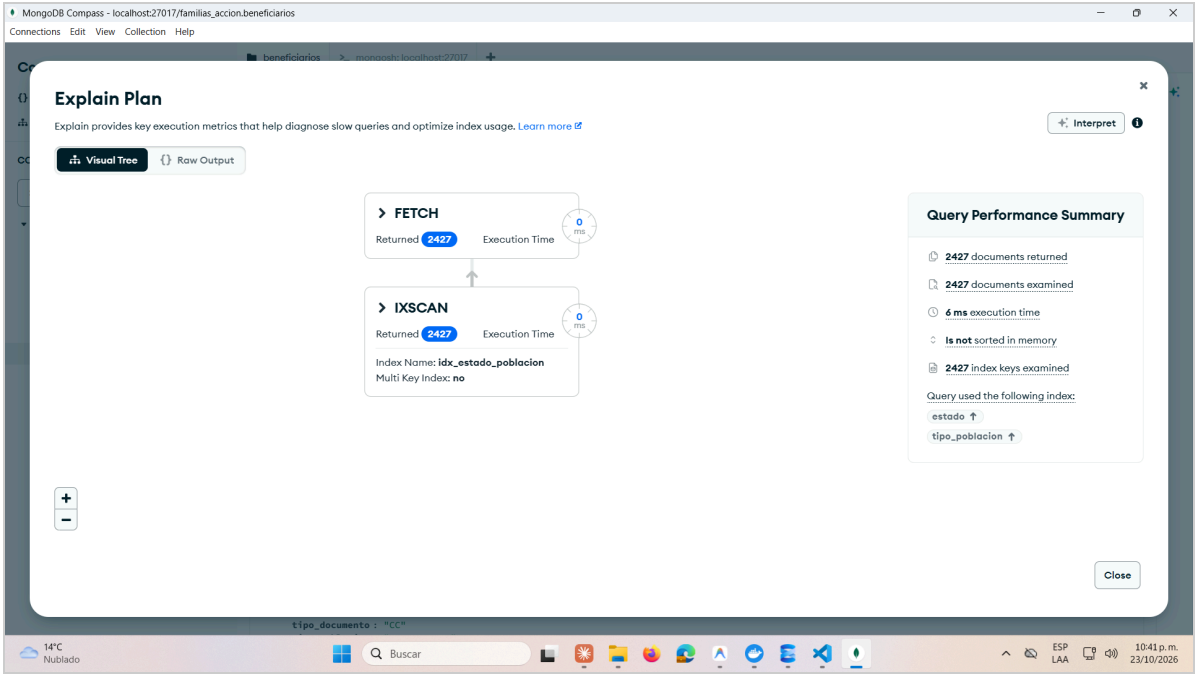
Pestaña Indexes: índices de la colección beneficiarios.



Nota. Elaboración propia. Captura de pantalla del proceso de configuración desarrollado por el autor.

Figura 32

Plan de ejecución (Explain Plan) de la consulta con índice.



Nota. Elaboración propia. Captura de pantalla del proceso de configuración desarrollado por el autor.

4.4 Análisis comparativo de resultados

Con el mismo CSV limpio, ambos motores deben coincidir (Tabla 4). La diferencia de una unidad en UNIDOS ACTIVO viene de la transacción del ejercicio 4, que reactivó al beneficiario 14 solo en CockroachDB, lo que confirma su `COMMIT`. SQL es más corto para uniones y funciones de ventana con esquema estable; MongoDB resulta más cómodo con documentos variables o arreglos anidados, y Compass permite armar el *pipeline* de forma visual.

Tabla 4
Resultados equivalentes en CockroachDB y MongoDB

Indicador	CockroachDB (SQL)	MongoDB (pipeline)	Coincide
Registros cargados	5.543	5.543	Sí
Bancarizados 30-49 años	2.763 de 2.939 (94,0 %)	2.763 de 2.939 (94,0 %)	Sí
SISBEN ACTIVO (registros / personas)	2.427 / 6.209	2.427 / 6.209	Sí
UNIDOS ACTIVO (registros / personas)	1.439 / 3.463	1.438 / 3.462	Difiere en 1

Nota. Elaboración propia.

4.5 Repositorio GitHub

Scripts, datos y evidencias están en un repositorio Git: https://github.com/Danielleon172025/repo-newsql-nosql-bigdata. El `README.md` explica cómo reproducir cada ejercicio (Figura 33); las figuras Figura 34, Figura 35 y Figura 36 muestran su creación, publicación y página principal.

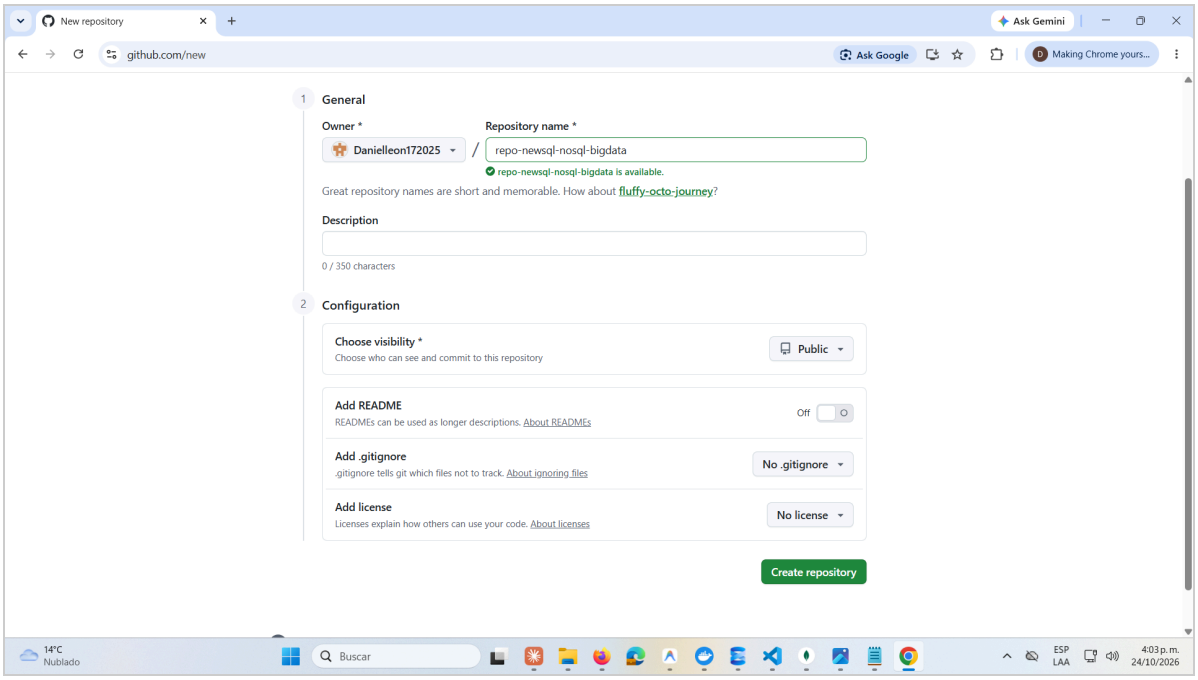
Figura 33
Estructura del repositorio

```
repo-newsql-nosql-bigdata/
|-- README.md                  Descripcion, requisitos y pasos de reproduccion
|-- data/        raw/ (CSV y JSON originales)   clean/ (CSV limpio)
|-- scripts/     python/ (limpieza)  newsql/ (SQL .sql + ejecutar_newsql.sh)
|                nosql/ (importacion y .js + ejecutar_nosql.sh)
|-- evidencias/  salidas de cada script e imagenes (img/)
`-- docs/        informe en PDF
```

Nota. Elaboración propia.

Figura 34

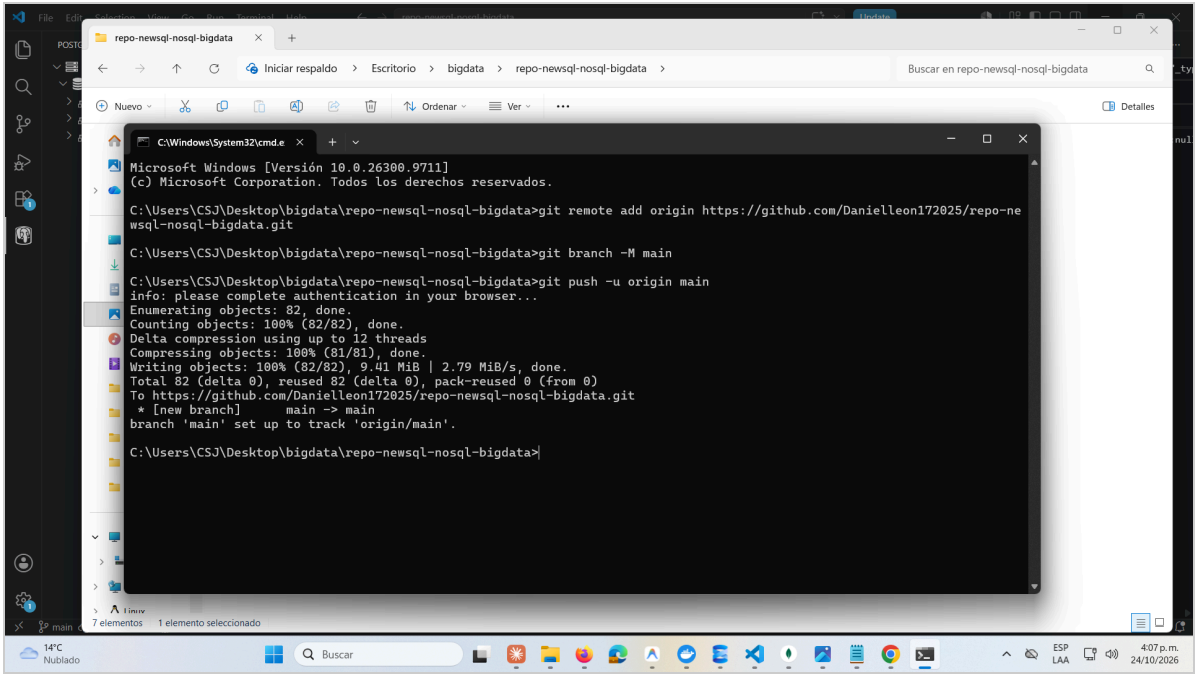
Creación del repositorio en GitHub.



Nota. Elaboración propia. Captura de pantalla del proceso de configuración desarrollado por el autor.

Figura 35

Publicación del repositorio con git push.



Nota. Elaboración propia. Captura de pantalla del proceso de configuración desarrollado por el autor.

Figura 36

Página principal del repositorio publicado en GitHub.

PENDIENTE — captura de pantalla
La dirección del repositorio en la barra del navegador, las carpetas `data`, `scripts`, `evidencias` y `docs`, y el README.
`evidencias/img/github_03_repositorio.png`

Nota. Elaboración propia. Captura de pantalla del proceso de configuración desarrollado por el autor.

4.6 Síntesis de comandos y pruebas ejecutadas

Tabla 5

Comandos, scripts y pruebas por herramienta

Herramienta	Script / comando	Prueba y resultado
Python	`01_recoleccion_limpieza.py`	Perfil y limpieza: 58 vacíos y 1.212 fechas centinela tratados.
CockroachDB	`01_esquema.sql`, `02_carga_beneficiarios.sql`	4 tablas creadas y 5.543 filas cargadas.
CockroachDB	`03_consultas.sql`, `04_transacciones_vistas.sql`, `05_verificar_tablas.sql`	7 consultas; COMMIT y ROLLBACK verificados; el índice se usa en `EXPLAIN`.
MongoDB	`mongoimport`, `01_importar.sh`	5.543 documentos y el JSON del SAB importados.
MongoDB	`02_crud_consultas.js`, `03_agregaciones.js`	CRUD y pipelines con resultados equivalentes a SQL.
MongoDB	`04_indices_rendimiento.js`	COLLSCAN (5.543) frente a IXSCAN (2.427); validador activo.

Nota. Elaboración propia.

5. Discusión

Los ejercicios recorrieron recolección, limpieza, carga, consulta, modificación e indexación en los dos modelos y dieron las mismas cifras. Cambian las garantías: `CHECK`, claves foráneas y transacciones multisentencia en CockroachDB; validación opcional y esquema flexible en MongoDB.

5.1 Limitaciones

(1) El cliente SQL fue Visual Studio Code y no Oracle SQL Developer, y no se trabajó sobre Oracle Database (sin PL/SQL). (2) Un nodo por motor y en modo inseguro: no se

evaluaron replicación, tolerancia a fallos ni seguridad. (3) Un solo municipio y 5.543 registros: válido para el procedimiento, no para medir rendimiento a escala. (4) Alta proporción de ND (91 % en etnia). (5) Las imágenes de scripts y consola se generaron de los archivos y salidas reales de ejecución.

6. Conclusiones

1. Se resolvieron ejercicios en NewSQL (CockroachDB) y NoSQL (MongoDB) sobre los mismos datos abiertos, y los resultados coinciden, lo que respalda la carga y las consultas.
2. CockroachDB mostró las garantías ACID de NewSQL (COMMIT, ROLLBACK, restricciones e índices) y se usó con SQL desde un cliente de PostgreSQL como Visual Studio Code.
3. MongoDB permitió un esquema flexible y agregaciones por etapas; un índice redujo los documentos examinados de 5.543 a 2.427.
4. Perfilar y limpiar antes de cargar dejó datos consistentes en ambos motores y permitió ver, por ejemplo, que ningún menor está bancarizado.
5. El repositorio permite repetir toda la práctica con los comandos del README.

7. Referencias

Cockroach Labs. (s. f.). *CockroachDB*. https://www.cockroachlabs.com/

Departamento para la Prosperidad Social. (2026). *Familias focalizadas primera fase del programa Familias en Acción etapa IV del municipio de Puerto Boyacá* [Conjunto de datos]. Datos Abiertos Colombia. https://www.datos.gov.co/

Galindo Rosales, R. (s. f.). *Nueva generación de motores de base de datos: NewSQL*. Medium. https://medium.com/@raulgalindorosales/nueva-generaci%C3%B3n-de-motores-de-base-de-datos-newsql-affadc477b22

Instituto Distrital de Gestión de Riesgos y Cambio Climático [IDIGER]. (2026). *SAB – Sistema de Alerta de Bogotá (Lluvia Minutal)* [Conjunto de datos]. Datos Abiertos Bogotá. https://datosabiertos.bogota.gov.co/dataset/sab-sistema-de-alerta-de-bogota-lluvia-minutal-idiger-bogota-d-c

Marr, B. (2016, 12 de febrero). Big data: 35 brilliant and free data sources for 2016. *Forbes*. https://www.forbes.com/sites/bernardmarr/2016/02/12/big-data-35-brilliant-and-free-data-sources-for-2016/

Microsoft. (s. f.). *Visual Studio Code*. https://code.visualstudio.com/

MongoDB, Inc. (s. f.). *MongoDB Compass*. https://www.mongodb.com/products/compass

Oracle. (s. f.). *Oracle SQL Developer*. https://www.oracle.com/database/sqldeveloper/

Universidad de Cundinamarca. (2026). *Guía: Manejo de DB NewSQL & NoSQL* [Guía de curso]. Especialización en Analítica de Datos.

Apéndice A. Scripts SQL y NoSQL utilizados

Scripts completos. `02_carga_beneficiarios.sql` (generado, 5.543 filas) se omite por su extensión y está en el repositorio.

scripts/python/01_recoleccion_limpieza.py

```
"""Recoleccion y limpieza del conjunto abierto 'Familias focalizadas - Familias en Accion
(Puerto Boyaca)' publicado en datos.gov.co.

Entrada : data/raw/familias_accion_puerto_boyaca.csv
Salidas : data/clean/beneficiarios_limpio.csv   (para mongoimport)
          scripts/newsql/02_carga_beneficiarios.sql (INSERT por lotes para CockroachDB)
          evidencias/00_perfilado_datos.txt     (perfil de calidad antes/despues)
Uso     : python 01_recoleccion_limpieza.py
"""
import csv
import collections
from pathlib import Path

RAIZ = Path(__file__).resolve().parents[2]
ENTRADA = RAIZ / "data" / "raw" / "familias_accion_puerto_boyaca.csv"
SALIDA_CSV = RAIZ / "data" / "clean" / "beneficiarios_limpio.csv"
SALIDA_SQL = RAIZ / "scripts" / "newsql" / "02_carga_beneficiarios.sql"
SALIDA_PERFIL = RAIZ / "evidencias" / "00_perfilado_datos.txt"

COLUMNAS = [
    "id", "bancarizado", "discapacidad", "estado", "etnia", "fecha_inscripcion",
    "genero", "nivel_escolaridad", "tipo_documento", "tipo_poblacion",
    "rango_beneficio_total", "rango_ultimo_beneficio", "fecha_ultimo_beneficio",
    "rango_edad", "titular", "cantidad_beneficiarios",
]


def normaliza_texto(valor):
    valor = (valor or "").strip().replace("\ufffd", "-")
    return valor.upper() if valor else "ND"


def normaliza_fecha(valor):
    """Las fechas '1900-01-01' son un centinela de 'sin dato' -> NULL."""
    valor = (valor or "").strip()
    return "" if valor in ("", "1900-01-01") else valor


def sql_literal(valor):
    return "NULL" if valor == "" else "'" + str(valor).replace("'", "''") + "'"


def main():
    with open(ENTRADA, encoding="utf-8-sig", newline="") as f:
        crudo = list(csv.DictReader(f))

    nulos_antes = sum(1 for r in crudo for v in r.values() if not (v or "").strip())
    limpio = []
    for i, r in enumerate(crudo, start=1):
        limpio.append({
            "id": i,
            "bancarizado": normaliza_texto(r["Bancarizado"]),
            "discapacidad": normaliza_texto(r["Discapacidad"]),
            "estado": normaliza_texto(r["EstadoBeneficiario"]),
            "etnia": normaliza_texto(r["Etnia"]),
            "fecha_inscripcion": normaliza_fecha(r["FechaInscripcionBeneficiario"]),
            "genero": normaliza_texto(r["Genero"]),
            "nivel_escolaridad": normaliza_texto(r["NivelEscolaridad"]),
            "tipo_documento": normaliza_texto(r["TipoDocumento"]),
            "tipo_poblacion": normaliza_texto(r["TipoPoblacion"]),
            "rango_beneficio_total": normaliza_texto(r["RangoBeneficioConsolidadoAsignado"]),
            "rango_ultimo_beneficio": normaliza_texto(r["RangoUltimoBeneficioAsignado"]),
            "fecha_ultimo_beneficio": normaliza_fecha(r["FechaUltimoBeneficioAsignado"]),
            "rango_edad": normaliza_texto(r["RangoEdad"]),
            "titular": normaliza_texto(r["Titular"]),
            "cantidad_beneficiarios": int(r["CantidadDeBeneficiarios"] or 1),
        })

    SALIDA_CSV.parent.mkdir(parents=True, exist_ok=True)
    with open(SALIDA_CSV, "w", encoding="utf-8", newline="") as f:
        w = csv.DictWriter(f, fieldnames=COLUMNAS)
        w.writeheader()
```

```
        w.writerows(limpio)

    with open(SALIDA_SQL, "w", encoding="utf-8") as f:
        f.write("USE familias_accion_db;\n\n")
        lote = 500
        for ini in range(0, len(limpio), lote):
            f.write("INSERT INTO beneficiarios (" + ", ".join(COLUMNAS) + ") VALUES\n")
            filas = []
            for r in limpio[ini:ini + lote]:
                vals = [str(r["id"])] + [sql_literal(r[c]) for c in COLUMNAS[1:-1]] + [str(r["cantidad_beneficiarios"])]
                filas.append("  (" + ", ".join(vals) + ")")
            f.write(",\n".join(filas) + ";\n\n")

    with open(SALIDA_PERFIL, "w", encoding="utf-8") as f:
        f.write(f"Registros leidos      : {len(crudo)}\n")
        f.write(f"Registros limpios     : {len(limpio)}\n")
        f.write(f"Celdas vacias (antes) : {nulos_antes}\n")
        f.write(f"Fechas centinela 1900 : {sum(1 for r in crudo if r['FechaUltimoBeneficioAsignado'].strip() == '1900-01-
01')}\n\n")
        for c in ["genero", "estado", "tipo_poblacion", "rango_edad", "bancarizado", "etnia"]:
            f.write(f"{c}: {dict(collections.Counter(r[c] for r in limpio).most_common())}\n")
    print(SALIDA_PERFIL.read_text(encoding="utf-8"))


if __name__ == "__main__":
    main()
```

scripts/newsql/01_esquema.sql

```
-- Ejercicio NewSQL 1: modelo relacional distribuido en CockroachDB
-- Dominio: beneficiarios del programa Familias en Accion (Puerto Boyaca)
DROP DATABASE IF EXISTS familias_accion_db CASCADE;
CREATE DATABASE familias_accion_db;
USE familias_accion_db;

-- Tablas de referencia (normalizacion de los dominios categoricos)
CREATE TABLE tipos_poblacion (
    codigo      STRING PRIMARY KEY,
    descripcion STRING NOT NULL
);

CREATE TABLE rangos_edad (
    codigo STRING PRIMARY KEY,
    edad_min INT NOT NULL,
    edad_max INT
);

-- Tabla de hechos
CREATE TABLE beneficiarios (
    id                      INT PRIMARY KEY,
    bancarizado             STRING NOT NULL CHECK (bancarizado IN ('SI','NO','ND')),
    discapacidad            STRING NOT NULL CHECK (discapacidad IN ('SI','NO','ND')),
    estado                  STRING NOT NULL CHECK (estado IN ('ACTIVO','NO ACTIVO')),
    etnia                   STRING NOT NULL,
    fecha_inscripcion       DATE,
    genero                  STRING NOT NULL,
    nivel_escolaridad       STRING NOT NULL,
    tipo_documento          STRING NOT NULL,
    tipo_poblacion          STRING NOT NULL REFERENCES tipos_poblacion (codigo),
    rango_beneficio_total   STRING NOT NULL,
    rango_ultimo_beneficio STRING NOT NULL,
    fecha_ultimo_beneficio DATE,
    rango_edad              STRING NOT NULL REFERENCES rangos_edad (codigo),
    titular                 STRING NOT NULL CHECK (titular IN ('SI','NO')),
    cantidad_beneficiarios INT NOT NULL CHECK (cantidad_beneficiarios >= 1),
    INDEX idx_estado_poblacion (estado, tipo_poblacion),
    INDEX idx_genero (genero)
);

-- Bitacora de cambios (se usa en el ejercicio de transacciones)
CREATE TABLE auditoria_cambios (
    id_cambio   UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    beneficiario_id INT NOT NULL REFERENCES beneficiarios (id) ON DELETE CASCADE,
    campo       STRING NOT NULL,
    valor_anterior STRING,
    valor_nuevo STRING,
    cambiado_en TIMESTAMPTZ NOT NULL DEFAULT now()
);

INSERT INTO tipos_poblacion VALUES
    ('SISBEN',      'Clasificado por el SISBEN'),
    ('UNIDOS',      'Red Unidos (pobreza extrema)'),
```

```
    ('DESPLAZADOS', 'Poblacion victima de desplazamiento'),
    ('ND',          'Sin dato');

INSERT INTO rangos_edad VALUES
    ('06-17', 6, 17), ('18-29', 18, 29), ('30-49', 30, 49), ('50-65', 50, 65), ('>65', 66, NULL);

SHOW TABLES;
```

scripts/newsql/03_consultas.sql

```
-- Ejercicio NewSQL 3: consultas SQL de analisis (SELECT, GROUP BY, JOIN, CTE, ventanas)
USE familias_accion_db;

-- 3.1 Conteo general y verificacion de la carga
SELECT count(*) AS total_registros FROM beneficiarios;

-- 3.2 Distribucion por estado y genero
SELECT estado, genero, count(*) AS total
FROM beneficiarios
GROUP BY estado, genero
ORDER BY estado, total DESC;

-- 3.3 Beneficiarios por tipo de poblacion (JOIN con tabla de referencia) y porcentaje
SELECT t.descripcion,
       count(*) AS total,
       round(100.0 * count(*) / sum(count(*)) OVER (), 2) AS porcentaje
FROM beneficiarios b
JOIN tipos_poblacion t ON t.codigo = b.tipo_poblacion
GROUP BY t.descripcion
ORDER BY total DESC;

-- 3.4 Bancarizacion por rango de edad (CASE + filtro HAVING)
SELECT rango_edad,
       count(*) AS total,
       count(*) FILTER (WHERE bancarizado = 'SI') AS bancarizados,
       round(100.0 * count(*) FILTER (WHERE bancarizado = 'SI') / count(*), 1) AS pct_bancarizados
FROM beneficiarios
GROUP BY rango_edad
HAVING count(*) > 100
ORDER BY rango_edad;

-- 3.5 CTE + funcion de ventana: ranking de rangos de beneficio dentro de cada tipo de poblacion
WITH conteo AS (
    SELECT tipo_poblacion, rango_beneficio_total, count(*) AS total
    FROM beneficiarios
    WHERE estado = 'ACTIVO'
    GROUP BY tipo_poblacion, rango_beneficio_total
)
SELECT tipo_poblacion, rango_beneficio_total, total,
       DENSE_RANK() OVER (PARTITION BY tipo_poblacion ORDER BY total DESC) AS posicion
FROM conteo
ORDER BY tipo_poblacion, posicion;

-- 3.6 Evolucion de inscripciones por anio (funciones de fecha)
SELECT extract(year FROM fecha_inscripcion) AS anio, count(*) AS inscritos
FROM beneficiarios
WHERE fecha_inscripcion IS NOT NULL
GROUP BY anio
ORDER BY anio;

-- 3.7 Consulta sobre la base de subredes de salud (semana 5-6): JOIN, JSONB y agregacion
SELECT s.nombre_sede, s.localidad,
       s.caracteristicas->>'camas_uci' AS camas_uci,
       count(t.tx_id) AS atenciones,
       round(avg(t.costo_estimado), 2) AS costo_promedio
FROM subred_newsql_db.sedes_prestadores s
JOIN subred_newsql_db.transacciones_atencion t ON t.sede_id = s.sede_id
GROUP BY s.nombre_sede, s.localidad, s.caracteristicas
ORDER BY atenciones DESC;
```

scripts/newsql/04_transacciones_vistas.sql

```
-- Ejercicio NewSQL 4: transacciones ACID, vistas, indices y plan de ejecucion
USE familias_accion_db;

-- 4.1 Transaccion: reactivar un beneficiario y registrar el cambio de forma atomica
SELECT id, estado FROM beneficiarios WHERE estado = 'NO ACTIVO' ORDER BY id LIMIT 1;

BEGIN;
  INSERT INTO auditoria_cambios (beneficiario_id, campo, valor_anterior, valor_nuevo)
```

```
  SELECT id, 'estado', estado, 'ACTIVO' FROM beneficiarios WHERE id = (SELECT min(id) FROM beneficiarios WHERE estado = 'NO
ACTIVO');
  UPDATE beneficiarios SET estado = 'ACTIVO'
  WHERE id = (SELECT min(id) FROM beneficiarios WHERE estado = 'NO ACTIVO');
COMMIT;

SELECT b.id, b.estado, a.campo, a.valor_anterior, a.valor_nuevo
FROM auditoria_cambios a JOIN beneficiarios b ON b.id = a.beneficiario_id;

-- 4.2 Transaccion revertida: el cambio es visible dentro de la transaccion y desaparece con ROLLBACK
BEGIN;
  UPDATE beneficiarios SET cantidad_beneficiarios = 99 WHERE id = 1;
  SELECT id, cantidad_beneficiarios AS dentro_de_la_transaccion FROM beneficiarios WHERE id = 1;
ROLLBACK;
SELECT id, cantidad_beneficiarios FROM beneficiarios WHERE id = 1;

-- 4.3 Restriccion de integridad: intento de insertar un valor no permitido (debe fallar)
-- INSERT INTO beneficiarios (id, bancarizado, discapacidad, estado, etnia, genero, nivel_escolaridad,
--   tipo_documento, tipo_poblacion, rango_beneficio_total, rango_ultimo_beneficio, rango_edad, titular, cantidad_beneficiarios)
-- VALUES (999999,'TALVEZ','NO','ACTIVO','ND','MUJER','ND','CC','SISBEN','0','0','30-49','SI',1);

-- 4.4 Vista materializada logica para el tablero de seguimiento
CREATE OR REPLACE VIEW v_resumen_poblacion AS
SELECT tipo_poblacion, estado, count(*) AS total,
       sum(cantidad_beneficiarios) AS personas_en_nucleo
FROM beneficiarios
GROUP BY tipo_poblacion, estado;

SELECT * FROM v_resumen_poblacion ORDER BY tipo_poblacion, estado;

-- 4.5 Plan de ejecucion: el indice secundario (estado, tipo_poblacion) evita un full scan
EXPLAIN SELECT count(*) FROM beneficiarios WHERE estado = 'ACTIVO' AND tipo_poblacion = 'SISBEN';

-- 4.6 Particion logica: rangos de la tabla distribuidos por el motor
SHOW RANGES FROM TABLE beneficiarios;
```

scripts/newsql/05_verificar_tablas.sql

```
SELECT table_name FROM familias_accion_db.information_schema.tables WHERE table_schema = 'public' ORDER BY table_name;
```

scripts/nosql/01_importar.sh

```
#!/usr/bin/env bash
# Importa los datos limpios a MongoDB (contenedor mi-mongodb). Las credenciales se toman de las
# variables de entorno del propio contenedor (MONGO_INITDB_ROOT_*), no se escriben en el repositorio.
set -e
export MSYS_NO_PATHCONV=1
cd "$(dirname "$0")"
C=mi-mongodb
docker cp ../../data/clean/beneficiarios_limpio.csv $C:/tmp/beneficiarios_limpio.csv
docker cp ../../data/raw/sab_tipo_sensores.json $C:/tmp/sab_tipo_sensores.json
# --ignoreBlanks omite los campos vacios (fechas sin dato) en lugar de guardar cadenas vacias
docker exec $C sh -c 'mongoimport -u "$MONGO_INITDB_ROOT_USERNAME" -p "$MONGO_INITDB_ROOT_PASSWORD" --authenticationDatabase
admin   --db familias_accion --collection beneficiarios --type csv --headerline --ignoreBlanks --drop --file
/tmp/beneficiarios_limpio.csv'
# El JSON del SAB es un unico objeto {"TipoSensores":[...]}: se importa como un documento
docker exec $C sh -c 'mongoimport -u "$MONGO_INITDB_ROOT_USERNAME" -p "$MONGO_INITDB_ROOT_PASSWORD" --authenticationDatabase
admin   --db SAB --collection sensores --drop --file /tmp/sab_tipo_sensores.json'
```

scripts/nosql/02_crud_consultas.js

```
// Ejercicio NoSQL 1: CRUD y consultas basicas en MongoDB (mongosh / Compass shell)
use("familias_accion");

print("--- 1.1 Total de documentos y un documento de ejemplo");
print(db.beneficiarios.countDocuments());
printjson(db.beneficiarios.findOne({}, { _id: 0 }));

print("--- 1.2 find con filtro y proyeccion: mujeres titulares, activas, de 30-49 anios");
printjson(db.beneficiarios.find(
  { genero: "MUJER", titular: "SI", estado: "ACTIVO", rango_edad: "30-49" },
  { _id: 0, id: 1, tipo_poblacion: 1, rango_beneficio_total: 1 }
).sort({ id: 1 }).limit(3).toArray());

print("--- 1.3 Operadores: personas con discapacidad O etnia indigena");
print(db.beneficiarios.countDocuments({ $or: [{ discapacidad: "SI" }, { etnia: "INDIGENA" }] }));

print("--- 1.4 $in y $gt: nucleos con mas de 3 integrantes en poblacion UNIDOS o DESPLAZADOS");
```

```
print(db.beneficiarios.countDocuments({ tipo_poblacion: { $in: ["UNIDOS", "DESPLAZADOS"] }, cantidad_beneficiarios: { $gt: 3 }
}));

print("--- 1.5 insertOne / updateOne / deleteOne");
db.beneficiarios.insertOne({ id: 999999, estado: "ACTIVO", genero: "MUJER", tipo_poblacion: "SISBEN", nota: "documento de prueba
(esquema flexible)", contacto: { telefono: "3000000000" } });
printjson(db.beneficiarios.findOne({ id: 999999 }, { _id: 0 }));
printjson(db.beneficiarios.updateOne({ id: 999999 }, { $set: { estado: "NO ACTIVO" }, $currentDate: { modificado: true }
}).modifiedCount);
printjson(db.beneficiarios.deleteOne({ id: 999999 }).deletedCount);

print("--- 1.6 updateMany: normalizar tipo_documento 'NO DEFINIDO' a 'ND'");
printjson(db.beneficiarios.updateMany({ tipo_documento: "NO DEFINIDO" }, { $set: { tipo_documento: "ND" } }).modifiedCount);

print("--- 1.7 Coleccion SAB.sensores (documentos con arreglo anidado)");
use("SAB");
printjson(db.sensores.find({}, { _id: 0, "TipoSensores": { $slice: 1 } }).toArray());
```

scripts/nosql/03_agregaciones.js

```
// Ejercicio NoSQL 2: aggregation pipeline (equivalente a GROUP BY / JOIN / ventanas de SQL)
use("familias_accion");

print("--- 2.1 $group: beneficiarios por tipo de poblacion y estado");
printjson(db.beneficiarios.aggregate([
  { $group: { _id: { poblacion: "$tipo_poblacion", estado: "$estado" }, total: { $sum: 1 }, personas: { $sum:
"$cantidad_beneficiarios" } } },
  { $sort: { "_id.poblacion": 1, "_id.estado": 1 } }
]).toArray());

print("--- 2.2 $match + $group + $project: % bancarizados por rango de edad");
printjson(db.beneficiarios.aggregate([
  { $group: { _id: "$rango_edad", total: { $sum: 1 }, bancarizados: { $sum: { $cond: [{ $eq: ["$bancarizado", "SI"] }, 1, 0] } }
} },
  { $project: { _id: 1, total: 1, bancarizados: 1, pct: { $round: [{ $multiply: [{ $divide: ["$bancarizados", "$total"] }, 100]
}, 1] } } },
  { $sort: { _id: 1 } }
]).toArray());

print("--- 2.3 $bucket: nucleos por tamanio");
printjson(db.beneficiarios.aggregate([
  { $bucket: { groupBy: "$cantidad_beneficiarios", boundaries: [1, 2, 3, 5, 10, 100], default: "otros", output: { nucleos: {
$sum: 1 } } } }
]).toArray());

print("--- 2.4 $group por anio de inscripcion (cadena ISO -> substring)");
printjson(db.beneficiarios.aggregate([
  { $match: { fecha_inscripcion: { $type: "string" } } },
  { $group: { _id: { $substr: ["$fecha_inscripcion", 0, 4] }, inscritos: { $sum: 1 } } },
  { $sort: { _id: 1 } }
]).toArray());

print("--- 2.5 $facet: varios resumenes en una sola consulta");
printjson(db.beneficiarios.aggregate([
  { $facet: {
      porGenero: [{ $group: { _id: "$genero", n: { $sum: 1 } } }],
      porEtnia: [{ $group: { _id: "$etnia", n: { $sum: 1 } } }, { $sort: { n: -1 } }, { $limit: 3 }]
  } }
]).toArray());

print("--- 2.6 SAB: $unwind + $group por localidad (sensores)");
use("SAB");
printjson(db.sensores.aggregate([
  { $unwind: "$TipoSensores" },
  { $group: { _id: "$TipoSensores.LOCALIDAD", sensores: { $sum: 1 } } },
  { $sort: { sensores: -1 } }, { $limit: 5 }
]).toArray());
```

scripts/nosql/04_indices_rendimiento.js

```
// Ejercicio NoSQL 3: indices y plan de ejecucion (explain)
use("familias_accion");

function resumen(plan) {
  const s = plan.queryPlanner.winningPlan;
  const etapa = (n) => n.inputStage ? n.stage + " > " + etapa(n.inputStage) : n.stage;
  return { etapas: etapa(s.queryPlan || s), docsExaminados: plan.executionStats.totalDocsExamined, devueltos:
plan.executionStats.nReturned };
}
```

```
db.beneficiarios.dropIndexes();
print("--- 3.1 Sin indice (COLLSCAN)");
printjson(resumen(db.beneficiarios.find({ estado: "ACTIVO", tipo_poblacion: "SISBEN" }).explain("executionStats")));

print("--- 3.2 Creacion de indice compuesto");
printjson(db.beneficiarios.createIndex({ estado: 1, tipo_poblacion: 1 }, { name: "idx_estado_poblacion" }));
printjson(db.beneficiarios.createIndex({ id: 1 }, { unique: true, name: "idx_id_unico" }));

print("--- 3.3 Con indice (IXSCAN)");
printjson(resumen(db.beneficiarios.find({ estado: "ACTIVO", tipo_poblacion: "SISBEN" }).explain("executionStats")));

print("--- 3.4 Indices existentes");
printjson(db.beneficiarios.getIndexes().map(i => i.name));

print("--- 3.5 Validacion de esquema opcional ($jsonSchema) en una coleccion nueva");
db.auditoria.drop();
db.createCollection("auditoria", { validator: { $jsonSchema: { bsonType: "object", required: ["beneficiario_id", "accion"],
properties: { beneficiario_id: { bsonType: "int" }, accion: { enum: ["alta", "baja", "cambio"] } } } } });
try { db.auditoria.insertOne({ beneficiario_id: 1, accion: "otro" }); } catch (e) { print("rechazado por validador (codigo " +
e.code + "): " + e.message); }
printjson(db.auditoria.insertOne({ beneficiario_id: 1, accion: "alta" }).acknowledged);
```